

Índice

1	Templates y Testing	1	16.1	Hashing	11
1.1	Template y archivos	1	16.2	KMP	12
1.2	Stress testing (diff)	1	16.3	Suffix Automata	12
1.3	Stress testing (checker)	1	16.4	Suffix array	12
2	Ideas	1	16.5	Aho-Corasick	12
3	Estructuras básicas	1	16.6	Suffix tree	13
3.1	Min queue	1	17	Geometría	13
3.2	Heap actualizable	1	17.1	Convex hull	13
4	Teoría de números	2	18	Utilidades	13
4.1	Cribas	2	18.1	Subset sum optimization.	13
4.2	Algoritmo extendido de Euclides	2	18.2	Bitsets de tamaño (casi) dinámico	14
4.3	Solución de ecuaciones diofánticas lineales	2	18.3	Bitwise ops	14
4.4	Funciones multiplicativas	2	18.4	Bitwise - builtin	14
4.5	Factorización Pollard Rho	3	18.5	Iterar	14
5	Combinatoria	3	18.6	Gospers' Hack	14
5.1	Números de Catalán	3	18.7	Subset Sum con bitset	14
5.2	Números de Narayana	4	18.8	SOS DP	14
5.3	Desarreglos (Derangements)	4	19	Máximo de funciones	14
5.4	Particiones Enteras	4	19.1	Li-Chao Tree	14
5.5	Permutaciones: Inversiones y Swaps	4			
6	Cálculo	4			
6.1	Transformada de Fourier	4			
7	Formulazas	5			
7.1	Lógica - Conjuntos - Bitwise	5			
7.2	Combinatoria	5			
7.3	Geometría	5			
8	Métodos numéricos	5			
8.1	Inversa de Matriz con Gauus-Jordan	5			
8.2	Sistemas de ecuaciones lineales	6			
8.3	Sistemas de ecuaciones modulo 2	6			
9	Sparse table	6			
10	Fenwick Tree	6			
11	Segment Tree	6			
11.1	Actualizaciones puntuales	6			
11.2	Actualizaciones sobre rangos	6			
11.3	Sparse segment tree	7			
12	Sqrt decomposition	7			
12.1	Algoritmo de MO	7			
13	DSU	7			
14	Grafos	7			
14.1	Caminos mínimos	7			
14.2	Árboles	8			
14.3	Máximo flujo	9			
14.4	SCC	10			
14.5	2-Sat	11			
15	Treap	11			
16	Strings	11			

TEMPLATES Y TESTING

► Template y archivos

template.h

```
1 #include <bits/stdc++.h>
2 #include <ext/pb_ds/assoc_container.hpp>
3 #include <ext/pb_ds/tree_policy.hpp>
4 using namespace std;
5 using namespace __gnu_pbds;
6 typedef int64_t ll;
7 typedef pair<int, int> pii;
8 typedef pair<ll, ll> pll;
9 typedef vector<int> vi;
10 typedef vector<ll> vll;
11 typedef vector<vi> vvi;
12 typedef vector<vll> vvll;
13 typedef vector<pii> vpii;
14 typedef vector<pll> vpll;
15 typedef tree<int, null_type, less<int>, rb_tree_tag,
    ↳ tree_order_statistics_node_update> ordered_set;
16 #define fi first
17 #define se second
18 #define pb push_back
19 #define all(x) (x).begin(), (x).end()
20 #define sz(x) (int)(x).size()
21 mt19937_64 gen(chrono::steady_clock::now().time_since_epoch().
    ↳ count());
22 uniform_int_distribution<ll> distr(1, LLONG_MAX);
23 const int MOD = 1e9 + 7;
24 template<typename A, typename B> ostream& operator<<(ostream
    ↳ &os, const pair<A, B> &p){return os << '(' << p.fi << ", "
    ↳ << p.se << ')';}
25 template<typename C, typename T = typename
    ↳ enable_if<!is_same<C, string>::value, typename
    ↳ C::value_type>::type>
26 ostream& operator<<(ostream &os, const C &v){string sep;
    ↳ for(const T &x : v) os << sep << x, sep = " "; return os;}
27 #define print(...) logger(#__VA_ARGS__, __VA_ARGS__)
28 template<typename ...Args>
29 void logger(string vars, Args&&... values){
30     cout << "[Debug]\n\t" << vars << " = ";
31     string d = "[";
32     (... , (cout << d << values, d = " ") );
33     cout << "]\n";
34 }
```

Ejecutar en la terminal de carpeta

```
1 for f in {a..n}; do cp template.cpp $f.cpp; done
```

► Stress testing (diff)

Para stress testing en problemas donde existe solo una respuesta, usamos *stress.sh*. Testeamos *code.cpp* contra *brute.cpp*. *gen.cpp* es el generador de casos. Este ultimo recibe de semilla el valor *i*.

stress.sh

```
1 set -e
2 g++ code.cpp -o code
3 g++ gen.cpp -o gen
4 g++ brute.cpp -o brute
5 for((i = 1; ; ++i)); do
6     ./gen $i > input_file
7     ./code < input_file > myAnswer
8     ./brute < input_file > correctAnswer
9     diff -Z myAnswer correctAnswer > /dev/null || break
10    echo "Passed test: " $i
11 done
12 echo "WA on the following test:"
13 cat input_file
14 echo "Your answer is:"
15 cat myAnswer
16 echo "Correct answer is:"
17 cat correctAnswer
```

Para extraer la semilla en gen.cpp leemos desde argv

```
1 int rnd(int a, int b){return a + rand() % (b - a + 1);}
2 int main(int argc, char* argv[]){
3     int seed = atoi(argv[1]);
4     srand(seed);
5     int n = rnd(1, 5);
6 }
```

► Stress testing (checker)

Usaremos el script similar al anterior *stress.sh*. Aquí el *brute.cpp* es opcional.

```
1 set -e
2 g++ code.cpp -o code
3 g++ gen.cpp -o gen
4 g++ brute.cpp -o brute
5 g++ checker.cpp -o checker
6 for((i = 1; ; ++i)); do
7     ./gen $i > input_file
8     ./code < input_file > myAnswer
9     ./brute < input_file > correctAnswer
10    ./checker > checker_log
11    echo "Passed test: " $i
12 done
```

Se ejecutara mientras el return value de checker sea 0. Ejemplo de la implementación de un checker

```
1 bool validAns(ifstream &fin, int n, int* arr){
2     fin>>n;
3     //aqui validariamos nuestra respuesta, fin sera el input
    ↳ stream que le pase
4     if(algunacosa) return true
5     else return false
6 }
7 int main(int argc, char * argv[]){
8     //input stream del caso
9     ifstream fin("input_file", ifstream::in);
10    //input stream de mi respuesta
11    ifstream ans("myAnswer", ifstream::in);
12    //input stream de respuesta bruta
13    ifstream cor("correctAnswer", ifstream::in);
14    //aqui fin puede procesar lo que interese del input original
15    fin>>n;
16    int arr[n];
17    for(int x = 0; x < n; x++) cin>>arr[x];
18    if(!validAns(ans, n, arr)) {
19        cout<<"WA x algun motivo, esto saldrá en cheker_log\n";
20        return -1;
21    }
22    return 0;
23 }
```

IDEAS

Ideas de Catalán. Ideas que se usan en las demostraciones de que algo se cuenta con Catalán y sirven para conteos similares:

1. *Partir en dos conjuntos y distribuir.* Usar la misma idea de la recurrencia de los números de Catalán: $(0, n - 1), (1, n - 2), \dots, (k, n - 1 - k)$.
2. *Reflejar los caminos malos.* Si queremos contar caminos monótonos que $(0, 0)$ y (n, m) , por debajo de una diagonal paralela a la que una dichas esquinas, en una cuadrícula de $n \times m$, contemos todos los caminos $\binom{n+m}{n}$ y restemos los caminos que pasan sobre la diagonal. Digamos que la diagonal es $y = x + k$, entonces los caminos malos pasan al menos una vez por $y = x + k + 1$: toma el primer punto $(x', x' + k + 1)$ sobre el que un camino pasa por $y = x + k + 1$ y refléjalo, nota que todos empiezan en $(x', x' + k + 1) - (x' + k + 1, x') = (-k - 1, k + 1)$.

Entonces los caminos malos son biyectivos a los caminos que van de $(-k - 1, k + 1)$ a (n, m) .

Optimización. Ideas para optimizar funciones.

1. *Fijar coordenadas secuencialmente.* Hay funciones en R^n que se pueden optimizar fijando la primera coordenada y resolviendo para R^{n-1} y así sucesivamente.

ESTRUCTURAS BÁSICAS

► Min queue

min_queue.cpp

```
1 template<typename T> struct min_stack{
2     stack<pair<T, T>> st;
3     min_stack(){}
4     min_stack(const T &MAXVAL){init(MAXVAL);}
5     void init(const T &MAXVAL){st.push({MAXVAL, MAXVAL});}
6     void push(const T &v){st.push({v, min(v, st.top().se)});}
7     T top(){return st.top().fi;}
8     void pop(){if(sz(st) > 1)st.pop();}
9     T minV(){return st.top().se;}
10    int size(){return sz(st) - 1;}
11    bool empty(){return size() == 0;}
12 };
13 template<typename T> struct min_queue{
14     min_queue(const T &MAXVAL){p_in.init(MAXVAL);
    ↳ p_out.init(MAXVAL);}
15     void push(const T &v){p_in.push(v);}
16     T front(){transfer(); return p_out.top();}
17     void pop(){transfer(); p_out.pop();}
18     int size(){return sz(p_in) + sz(p_out);}
19     T minV(){return min(p_in.minV(), p_out.minV());}
20     bool empty(){return size() == 0;}
21     void transfer(){
22         if(sz(p_out)) return;
23         while(sz(p_in)){p_out.push(p_in.top()); p_in.pop();}
24     }
25     min_stack<T> p_in, p_out;
26 };
```

► Heap actualizable

updateable_heap.cpp

```
1 template<class TPriority, class TKey> class UpdateableHeap{
2 public:
3     UpdateableHeap(){
4         TPriority a;
5         TKey b;
6         nodes.clear(); nodes.pb({a, b});
7     }
8     pair<TPriority, TKey> top(){return nodes[1];}
9     void pop(){
10        if(sz(nodes) == 1) return;
11        TKey k = nodes[1].second;
12        swap_nodes(1, sz(nodes) - 1);
13        nodes.pop_back();
14        position.erase(k);
15        heapify(1);
16    }
17    void insert_or_update(const TPriority &p, const TKey &k){
18        int pos;
19        if(is_inserted(k)){
20            pos = position[k];
21            nodes[pos].first += p;
22        } else {
23            position[k] = pos = sz(nodes);
24            nodes.pb({p, k});
25            heapify(pos);
26        }
27    }
28    bool is_inserted(const TKey &k){return position.count(k);}
29    int size(){return sz(nodes) - 1;}
30    void erase(const TKey &k){
31        if(!is_inserted(k)) return;
32        int pos = position[k];
33        swap_nodes(pos, sz(nodes) - 1);
```

```

33     nodes.pop_back();
34     position.erase(k);
35     heapify(pos);
36 }
37 private:
38     vector<pair<TPriority, TKey>> nodes;
39     map<TKey, int> position;
40     void heapify(int pos){
41         if(pos >= sz(nodes)) return;
42         while(1 < pos && nodes[pos / 2] <= nodes[pos]){
43             swap_nodes(pos / 2, pos);
44             pos /= 2;
45         }
46         int l = pos * 2, r = pos * 2 + 1, maxi = pos;
47         if(l < sz(nodes) && nodes[l] > nodes[maxi]) maxi = l;
48         if(r < sz(nodes) && nodes[r] > nodes[maxi]) maxi = r;
49         if(maxi != pos){
50             swap_nodes(pos, maxi);
51             heapify(maxi);
52         }
53     }
54     void swap_nodes(int a, int b){
55         position[nodes[a].se] = b;
56         position[nodes[b].se] = a;
57         swap(nodes[a], nodes[b]);
58     }
59 };

```

TEORÍA DE NÚMEROS

► Cribas

T: $O(n \log \log n)$ | **M:** $O(n)$ |

```

1 void sieve(int n, vi &primes){
2     primes.clear(); if(n < 2) return;
3     vector<bool> nprime(n + 1);
4     nprime[0] = nprime[1] = 1;
5     for(ll i = 3; i * i <= n; i += 2) if(!nprime[i])
6         for(ll j = i * i; j <= n; j += 2 * i) nprime[j] = 1;
7     primes.pb(2);
8     for(int i = 3; i <= n; i += 2) if(!nprime[i]) primes.pb(i);
9 }

```

sieve.cpp

T: $O((\sqrt{b} + b - a) \log \log \sqrt{b})$ | **M:** $O(\sqrt{b} + b - a)$ | Encuentra los primos en $[a, b]$ con $b - a \leq K \cdot 10^6$

```

1 void range_sieve(ll a, ll b, vll &primes){
2     a = max(a, 0ll);
3     b = max(b, 0ll);
4     ll len = b - a + 1;
5     vi sqrt_primes;
6     sieve(sqrt(b) + 1, sqrt_primes);
7     vector<char> nprime(len);
8     primes.clear();
9     for(ll p : sqrt_primes){
10         ll ini = p * max(p, (a + p - 1) / p);
11         for(ll m = ini; m <= b; m += p) nprime[m - a] = 1;
12     }
13     for(ll i = 0; i < len; ++i)
14         if(!nprime[i] || i + a < 2) primes.pb(i + a);
15 }

```

range_sieve.cpp

T: $O((n + \sqrt{n}) \log \log \sqrt{n})$ | **M:** $O(\sqrt{n} + S)$ | Cuenta primos menores o iguales a n .

```

1 int count_primes(int n){
2     if(n < 2) return 0;
3     const int S = sqrt(n);
4     vi sqrt_primes;
5     sieve(sqrt(n) + 1, sqrt_primes);
6     int ans = 0;
7     vector<char> nprime(S + 1);
8     for(int ini = 0; ini <= n; ini += S){
9         fill(all(nprime), 0);
10        for(int p : sqrt_primes){

```

segmented_sieve.cpp

```

11        ll m = 1ll * p * max(p, (ini + p - 1) / p) - ini;
12        for(; m <= S; m += p) nprime[m] = 1;
13    }
14    for(int i = 0; i < S && i + ini <= n; ++i)
15        if(!nprime[i] && 1 < i + ini) ans++;
16    } return ans;
17 }

```

T: $O(n)$ | **M:** $O(n)$ | Para calcular funciones multiplicativas difíciles de adaptar en la criba de Eratóstenes es mejor la criba lineal.

```

1 void linear_sieve(int n, vi &primes){
2     primes.clear();
3     if(n < 2) return;
4     vi lp(n + 1);
5     for(ll i = 2; i <= n; ++i){
6         if(!lp[i]) primes.pb(lp[i] = i);
7         for(int j = 0; i * primes[j] <= n; ++j){
8             lp[i * primes[j]] = primes[j];
9             if(primes[j] == lp[i]) break;
10        }
11    }
12 }

```

linear_sieve.cpp

► Algoritmo extendido de Euclides

T: $O(\log \min(a, b))$ | **M:** $O(1)$ | Encuentra una solución de la ecuación $ax + by = \gcd(a, b)$.

```

1 ll ext_gcd(ll a, ll b, ll &x, ll &y){
2     if(!b) return x = 1, y = 0, a;
3     ll g = ext_gcd(b, a % b, x, y);
4     return y -= a / b * x, g;
5 }

```

extended_gcd.cpp

► Solución de ecuaciones diofánticas lineales

T: $O(\log \min(a, b))$ | **M:** $O(1)$ | Encuentra o cuenta soluciones de la ecuación $ax + by = c$.

```

1 bool find_sol(ll a, ll b, ll c, ll &x, ll &y, ll &g){
2     g = ext_gcd(abs(a), abs(b), x, y);
3     if(c % g) return false;
4     x *= c / g;
5     y *= c / g;
6     if(a < 0) x = -x;
7     if(b < 0) y = -y;
8     return true;
9 }
10 void change_sol(ll &x, ll &y, ll a, ll b, ll cnt, ll g = 1){
11     x += cnt * b / g;
12     y -= cnt * a / g;
13 }
14 ll cntsol(ll a, ll b, ll c, ll minx, ll maxx, ll miny, ll maxy){
15     ll x, y, g;
16     if(!find_sol(a, b, c, x, y, g)) return 0;
17     a /= g; b /= g;
18     int sign_a = a > 0 ? +1 : -1; int sign_b = b > 0 ? +1 : -1;
19     change_sol(x, y, a, b, (minx - x) / b);
20     if(x < minx) change_sol(x, y, a, b, sign_b);
21     if(x > maxx) return 0;
22     ll lx1 = x;
23     change_sol(x, y, a, b, (maxx - x) / b);
24     if(x > maxx) change_sol(x, y, a, b, -sign_b);
25     ll rx1 = x;
26     change_sol(x, y, a, b, -(miny - y) / a);
27     if(y < miny) change_sol(x, y, a, b, -sign_a);
28     if(y > maxy) return 0;
29     ll lx2 = x;
30     change_sol(x, y, a, b, -(maxy - y) / a);
31     if(y > maxy) change_sol(x, y, a, b, sign_a);
32     ll rx2 = x;
33     if(lx2 > rx2) swap(lx2, rx2);
34     ll lx = max(lx1, lx2); ll rx = min(rx1, rx2);
35     if(lx > rx) return 0;
36     return (rx - lx) / abs(b) + 1;
37 }

```

diofantine_equations.cpp

► Funciones multiplicativas

T: $O(n \log \log n)$ | **M:** $O(n)$ | Calcula $\phi(i)$ para todo $i \in [1, n]$.

```

1 vi calc_phi(int n){
2     vi phi(n + 1);
3     for(int i = 0; i <= n; ++i) phi[i] = i & 1 ? i : i / 2;
4     for(int i = 3; i <= n; i += 2) if(phi[i] == i)
5         for(int j = i; j <= n; j += i) phi[j] -= phi[j] / i;
6     return phi;
7 }

```

phi_euler.cpp

Para p primo y $a, b, n, k \in \mathbb{Z}^+$ se cumple

$$\phi(p^k) = p^{k-1}(p-1),$$

$$\phi(ab) = \phi(a)\phi(b) \frac{\gcd(a, b)}{\phi(\gcd(a, b))},$$

$$\phi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right),$$

$$n = \sum_{d|n} \phi(d).$$

Si $G_d = \{x : 1 \leq x \leq n, \gcd(x, n) = d\}$ entonces $|G_d| = \phi(\frac{n}{d})$. Si $\gcd(x, m) = 1$ entonces $x^{\phi(m)} \equiv 1 \pmod{m}$. Si $\gcd(x, m) \neq 1$ entonces $x^n \equiv x^{\phi(m) + [n \bmod \phi(m)]} \pmod{m}$.

Función σ_x . Si p es primo y $a > 0$ entonces

$$\sigma_x(p^a) = \sum_{i=0}^a p^{ix} = \frac{p^{x(a+1)} - 1}{p^x - 1}.$$

Si $\gcd(n, m) = 1$ entonces $\sigma_x(nm) = \sigma_x(n)\sigma_x(m)$. En particular, $\sigma_0(n)$ es la cantidad de divisores de n y $\sigma_1(n)$ es la suma de los divisores de n . **NOTA:** varios problemas que involucran suma de divisores, suma de suma de divisores, suma de la cantidad de divisores de los divisores, etc, pueden resolverse con argumentos combinatorios para cada primo y luego multiplicando los resultados. Por ejemplo, si $n = p_1^{a_1} p_2^{a_2} \dots p_k^{a_k}$ entonces

$$\sum_{d|n} d = \prod_{i=1}^k \sigma_1(p_i^{a_i}) = \prod_{i=1}^k \frac{p_i^{a_i+1} - 1}{p_i - 1},$$

$$\sum_{d|n} \sigma_0(d) = \prod_{i=1}^k \sum_{j=0}^{a_i} (j+1) = \prod_{i=1}^k \frac{(a_i+1)(a_i+2)}{2}.$$

T: $O(n)$ | **M:** $O(n)$ | Criba lineal para calcular σ_0 o σ_1 .

```

1 void linear_sigma(int n, vi &lp, vll &sigma1){
2     vi primes;
3     lp.resize(n + 1, 0);
4     sigma1.resize(n + 1, 0);
5     vll sum_p(n + 1, 0);
6     sigma1[1] = 1;
7     for(ll i = 2; i <= n; ++i){
8         if(lp[i] == 0){
9             lp[i] = i;
10            primes.pb(i);
11            sigma1[i] = i + 1;
12            sum_p[i] = i + 1;

```

sigma_fnc.cpp

```

13     }
14     for(int p : primes){
15         if(p > lp[i] || i * p > n) break;
16         lp[i * p] = p;
17         if(lp[i] == p){
18             sum_p[i*p] = sum_p[i] * p + 1;
19             sigma1[i*p] = sigma1[i] / sum_p[i] * sum_p[i*p];
20             break;
21         } else {
22             sum_p[i * p] = p + 1;
23             sigma1[i * p] = sigma1[i] * (p + 1);
24         }
25     }
26 }
27 }

```

Función de Möbius. Para un entero $n \geq 1$, $\mu(n)$ se define como:

$$\mu(n) = \begin{cases} 1, & \text{si } n = 1, \\ 0, & \text{si } n \text{ tiene un factor primo al cuadrado,} \\ (-1)^k, & \text{si } n \text{ es el producto de } k \text{ primos distintos.} \end{cases}$$

T: $O(n)$ | **M:** $O(n)$ | *Criba lineal para calcular la función de Möbius.*

```

1 void linear_mobius(int n, vi &primes, vi &mu) {
2     vi lp(n + 1);
3     mu.assign(n + 1, 0);
4     mu[1] = 1;
5     for(ll i = 2; i <= n; ++i){
6         if(!lp[i]){
7             primes.pb(lp[i] = i);
8             mu[i] = -1;
9         }
10        for(int j = 0; i * primes[j] <= n; ++j){
11            lp[i * primes[j]] = primes[j];
12            if(primes[j] == lp[i]){
13                mu[i * primes[j]] = 0;
14                break;
15            } else mu[i * primes[j]] = -mu[i];
16        }
17    }
18 }

```

► Factorización Pollard Rho

Complejidad: $O(\sqrt[4]{n})$. Inicializar con PollardRho::init() y para factorizar un número PollardRho::factorize(n).

```

1 //COPIADO Y PEGADOPORCUESTIONES DE TIEMPO
2 namespace PollardRho {
3     mt19937 rnd(chrono::steady_clock::now()
4         .time_since_epoch().count());
5     const int P = 1e6 + 9;
6     ll seq[P];
7     int primes[P], spf[P];
8     inline ll add_mod(ll x, ll y, ll m){
9         return (x += y) < m ? x : x - m; }
10    inline ll mul_mod(ll x, ll y, ll m) {
11        ll res = __int128(x) * y % m;
12        return res;
13        // ll res = x * y - (ll)((long double)x * y / m + 0.5) *
14        // ↳ m;
15        // return res < 0 ? res + m : res;
16    }
17    inline ll pow_mod(ll x, ll n, ll m) {
18        ll res = 1 % m;
19        for (; n; n >>= 1) {
20            if (n & 1) res = mul_mod(res, x, m);
21            x = mul_mod(x, x, m);
22        }
23        return res;
24    }
25    // 0(it * (logn)^3, it = number of rounds performed
26    inline bool miller_rabin(ll n) {
27        if (n<=2 || (n & 1 ^ 1)) return (n==2);
28        if (n < P) return spf[n] == n;

```

```

28        ll c, d, s = 0, r = n - 1;
29        for (; !(r & 1); r >= 1, s++) {}
30        // each iteration is a round
31        for(int i=0; primes[i]<n && primes[i]<32; i++){
32            c = pow_mod(primes[i], r, n);
33            for(int j = 0; j < s; j++){
34                d = mul_mod(c, c, n);
35                if(d==1&&c!=1&&c!=n-1) return 0;
36                c = d;
37            } if (c!=1) return 0;
38        } return 1;
39    }
40    void init() {
41        int cnt = 0;
42        for(int i = 2; i < P; i++){
43            if(!spf[i]) primes[cnt++] = spf[i]=i;
44            for(int j=0,k;(k=i*primes[j])<P;j++){
45                spf[k] = primes[j];
46                if (spf[i] == spf[k]) break;
47            }
48        }
49        // returns 0(n^(1/4))
50        ll pollard_rho(ll n) {
51            while(1){
52                ll x=rnd()%n,y=x,c=rnd()%n,u=1,v,t=0;
53                ll *px = seq, *py = seq;
54                while(1){
55                    *py++ = y = add_mod(mul_mod(y, y, n), c, n);
56                    *py++ = y = add_mod(mul_mod(y, y, n), c, n);
57                    if((x = *px++) == y) break;
58                    v = u;
59                    u = mul_mod(u, abs(y - x), n);
60                    if(!u) return __gcd(v, n);
61                    if(++t == 32){
62                        t = 0;
63                        if((u = __gcd(u,n))>1 && u<n)
64                            return u;
65                    }
66                }
67                if(t && (u = __gcd(u, n)) > 1 && u<n)
68                    return u;
69            }
70        }
71        vector<ll> factorize(ll n) {
72            if(n == 1) return vector<ll>{};
73            if(miller_rabin(n)) return vector<ll>{n};
74            vector<ll> v, w;
75            while(n > 1 && n < P){
76                v.pb(spf[n]); n /= spf[n];
77            }
78            if(n >= P){
79                ll x = pollard_rho(n);
80                v = factorize(x);
81                w = factorize(n / x);
82                v.insert(v.end(), all(w));
83            } return v;
84        }
85    }
86 }

```

COMBINATORIA

► Números de Catalán

Se puede calcular con $C_0 = C_1 = 1$,

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k} = \frac{1}{n+1} \binom{2n}{n} = \frac{4n+2}{n+2} C_{n-1}, \quad n \geq 2.$$

El n -ésimo número de Catalán C_n cuenta

- La cantidad de secuencias de paréntesis balanceadas de longitud $2n$.
- La cantidad de maneras distintas de agrupar $n+1$ factores con paréntesis.
- La cantidad de triangulaciones de un polígono convexo de $n+2$

lados.

- La cantidad de maneras de unir $2n$ puntos en una circunferencia con cuerdas sin que ningún par se corte.
- La cantidad de árboles binarios completos con n nodos *internos* no isomorfos. Los nodos *internos* son aquellos con dos hijos.
- La cantidad de árboles binarios enraizados completos no isomorfos con $n+1$ hojas.
- La cantidad de árboles enraizados planos no isomorfos con $n+1$ nodos.
- La cantidad de árboles binarios no isomorfos con exactamente n nodos.
- La cantidad de caminos monótonos en un tablero de $n \times n$ que van de $(0,0)$ a (n,n) sin que cruce la diagonal que une $(0,0)$ con (n,n) .
- La cantidad de permutaciones de tamaño n que son ordenables con una pila (mientras $\text{top}() \leq x$, $\text{pop}()$. Luego $\text{push}(x)$. Al final haz $\text{pop}()$ de los elementos restantes de la pila). Equivalentemente la cantidad de permutaciones que no contienen el patrón 231: no existen índices $i < j < k$ tales que $a_k < a_i < a_j$.
- La cantidad de permutaciones de tamaño n que no contienen el patrón 123: no existen índices $i < j < k$ tales que $a_i < a_j < a_k$.
- La cantidad de particiones no cruzadas de un conjunto de tamaño n .
- La cantidad de maneras de cubrir una escalera con n escalones, con la altura del i -ésimo escalón siendo i , mediante n rectángulos.
- La cantidad de maneras de unir n cuadros de 1×1 tales que cada cuadro tenga a otro cuadro adyacente a sus lados y, cada columna de cuadros tenga una altura absoluta mayor o igual a la altura absoluta a la columna previa. Cada uno de estos polígonos tiene un perímetro de $2n+2$.

Triángulo de Catalán. Sean $n, k \in \mathbb{Z}_{\geq 0}$, definamos

$$C_{n,k} = \begin{cases} 0, & n < k \text{ o } n, k < 0, \\ 1, & n = k = 0, \\ C_{n,k-1} + C_{n-1,k}, & k \leq n. \end{cases}$$

Entonces $C_{n,n} = C_n = \sum_{k=1}^{n-1} C_{n-1,k}$. El número $C_{n,k}$ se puede interpretar como la cantidad de caminos desde (n,k) del Triángulo de Catalán hasta $(0,0)$. Por lo que tiene sentido que $C_{n,n} = C_n$, dado que $C_{n,n}$ es la cantidad de caminos monótonos desde (n,n) a $(0,0)$ en un tablero de $n \times n$ que no cruzan la diagonal que une $(0,0)$ y (n,n) .

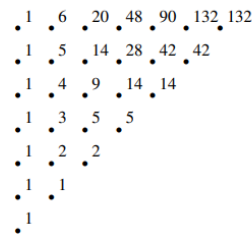


Figura 5.1: Triángulo de Catalán hasta $n = 6$.

► **Números de Narayana**

Sean $n, k \in \mathbb{Z}^+$ con $k \leq n$, los números de Narayana se definen como

$$N(n, k) = \frac{1}{n} \binom{n}{k} \binom{n}{k-1}.$$

Se cumple que

$$N(n, k) = N(n, n - k + 1),$$
$$\sum_{k=1}^n N(n, k) = C_n,$$

donde C_n es el n -ésimo número de Catalán. El número $N(n, k)$ cuenta

- La cantidad de secuencias de paréntesis balanceadas con $2n$ paréntesis y k distintos anidamientos, es decir, k ocurrencias de la subcadena $()$.
- La cantidad de caminos distintos desde $(0, 0)$ a $(2n, 0)$ dando pasos hacia arriba o abajo y siempre avanzando una unidad en cada paso (diagonales), de manera que haya exactamente k picos. Equivalentemente a la cantidad de maneras de ordenar n 1's y n (-1) 's en una secuencia a_i tales que si S_i es la suma parcial de a_i , entonces $\{S_i\}$ tiene exactamente k máximos locales.

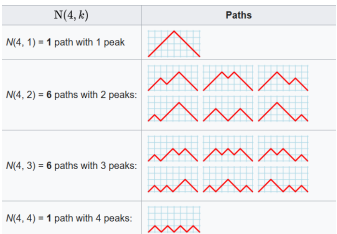


Figura 5.2: Caminos de $(0, 0)$ a $(8, 0)$ que cuenta $N(4, k)$.

- La cantidad de árboles enraizados con $n + 1$ nodos y k hojas.
- La cantidad de particiones no cruzadas de un conjunto de tamaño n en exactamente k subconjuntos.

► **Desarreglos (Derangements)**

$!n = D_n = D_{n,0}$: Cantidad de permutaciones de n elementos sin ningún punto fijo ($\pi(i) \neq i, \forall i$).

Fórmulas y Recurrencias:

- $D_n = (n - 1)(D_{n-1} + D_{n-2})$ con $D_0 = 1, D_1 = 0$.
- $D_n = nD_{n-1} + (-1)^n$ para $n \geq 1$.
- (PIE): $D_n = n! \sum_{i=0}^n \frac{(-1)^i}{i!} = n! \left(1 - \frac{1}{1!} + \frac{1}{2!} - \dots + \frac{(-1)^n}{n!} \right)$.
- $D_n = \left\lfloor \frac{n!+1}{e} \right\rfloor = \text{round} \left(\frac{n!}{e} \right)$ para $n \geq 1$.
- $\lim_{n \rightarrow \infty} \frac{D_n}{n!} = \frac{1}{e} \approx 0.367879$.
- Exactamente k puntos fijos:** $D_{n,k} = \binom{n}{k} D_{n-k}$.
- $\sum_{k=0}^n \binom{n}{k} D_k = n!$.

Valores iniciales ($n = 0 \dots 6$): 1, 0, 1, 2, 9, 44, 265.

Objetos que cuenta D_n :

- Repartos de n objetos a n destinatarios donde nadie recibe el correcto (ej. cartas en sobres).
- Matrices de permutación de $n \times n$ con ceros en la diagonal principal.
- Grafos dirigidos 1-regulares de n vértices sin autobucles (self-loops).

► **Particiones Enteras**

T: $O(\frac{13\sqrt{n}}{n})$ | **M:** $O(n)$ | Genera todas las particiones de un número n como suma de enteros positivos en orden no creciente.

```
1 void generar(int n, int maximo, vi &actual){
2   if(n == 0){// Aqui trabajar con actual
3     return; }
4   for(int i = min(n, maximo); i >= 1; --i){
5     actual.pb(i);
6     generar(n - i, i, actual);
7     actual.pop_back();
8   }
9 }
```

La función de partición $p(n)$, que cuenta el número de formas distintas de escribir n como suma de enteros positivos sin importar el orden,

n	1	2	3	4	5	6	7	8
$p(n)$	1	2	3	5	7	11	15	22
n	9	10	11	12	13	14	15	16
$p(n)$	30	42	56	77	101	135	176	231

Tabla 1: Particiones enteras

Se tiene la siguiente aproximación asintótica para valores grandes de n , conocida como la **fórmula de Hardy-Ramanujan**:

$$p(n) \sim \frac{1}{4n\sqrt{3}} \exp \left(\pi \sqrt{\frac{2n}{3}} \right)$$

► **Permutaciones: Inversiones y Swaps**

Sea P una permutación de tamaño n sobre $\{0, \dots, n - 1\}$:

- Inversión:** Par de índices (i, j) tal que $i < j$ y $P[i] > P[j]$.
- Swaps Adyacentes mín.:** Iguales a la cantidad de **inversiones**.

- Swaps Generales mín.:** $n - c(P)$, donde $c(P)$ es el número de ciclos en $iP[i]$.
- Paridad:** $\#Inversiones \equiv n - c(P) \pmod{2}$.

Distancia entre permutaciones $A \rightarrow B$: Para hallar el costo de transformar A en B , se mapean los elementos de A a las posiciones que deben ocupar en B :

- Construir **pos** tal que **pos** $[B[i]] = i$.
- Generar $P[i] = \text{pos}[A[i]]$.
- La distancia en swaps adyacentes/generales es el número de inversiones/ $n - c(P)$ de P .

Peores casos y distancias máximas ($A \rightarrow B$):

- Inversiones (Swaps ady.):** $B = \text{reverse}(A)$, Costo = $n(n - 1)/2$.
- Swaps Generales:** $B = \text{shift_circular}(A)$ (1 solo ciclo), Costo = $n - 1$.

T: $O(n \log n)$ | **M:** $O(n)$ |

```
1 long long merge_inv(vector<int>& a, int l, int r, vector<int>&
   tmp) {
2   if (r - l <= 1) return 0;
3   int m = l + (r - l) / 2;
4   long long inv = merge_inv(a, l, m, tmp) + merge_inv(a, m,
   r, tmp);
5   int i = l, j = m, k = l;
6   while (i < m && j < r) {
7     if (a[i] <= a[j]) tmp[k++] = a[i++];
8     else { tmp[k++] = a[j++]; inv += (m - i); }
9   }
10  while (i < m) tmp[k++] = a[i++];
11  while (j < r) tmp[k++] = a[j++];
12  for (int t = l; t < r; t++) a[t] = tmp[t];
13  return inv;
14 }
15 long long contar_inversiones(vector<int> a) {
16   vector<int> tmp(a.size());
17   return merge_inv(a, 0, a.size(), tmp);
18 }
```

CÁLCULO

► **Transformada de Fourier**

FFT. Complejidad: Tiempo $O(n \log n)$ - Memoria extra $O(n \log n)$, donde n es el grado del polinomio P .

```
1 using comp = complex<double>;
2 const double PI = acos(-1);
3 vector<comp> FFT(vector<comp> &P, bool inversa){
4   int n = sz(P);
5   if(n == 1) return P;
6   vector<comp> Pe, Po;
7   for(int i = 0; i < n; ++i)
8     if(i % 2) Po.pb(P[i]);
9     else Pe.pb(P[i]);
10  vector<comp> eval_Pe = FFT(Pe, inversa);
11  vector<comp> eval_Po = FFT(Po, inversa);
12  vector<comp> eval(n);
13  double angulo = 2*PI / n * (inversa? -1 : 1);
14  comp w(1), w_n(cos(angulo), sin(angulo));
15  for(int i = 0; i < n / 2; ++i){
16    eval[i] = eval_Pe[i] + w * eval_Po[i];
17    eval[i+n/2] = eval_Pe[i] - w*eval_Po[i];
18    if(inversa){eval[i]/=2; eval[i+n/2]/=2;}
19    w *= w_n;
20  } return eval;
21 }
```

Multiplicar polinomios. Complejidad: Tiempo $O(n \log n)$ - Memoria extra $O(n \log n)$, donde n es el grado máximo del polinomio A o B .

```
1 vi multiplicar(vi A, vi B){
2   vector<comp> cA(all(A)), cB(all(B));
3   int n = 1;
4   while(n < sz(A) + sz(B)) n *= 2;
5   cA.resize(n);
6   cB.resize(n);
7   vector<comp> val_A = FFT(cA, false);
8   vector<comp> val_B = FFT(cB, false);
9   for(int i=0; i<n; ++i) val_A[i] *= val_B[i];
10  val_A = FFT(val_A, true);
11  vi res(n);
12  for(int i = 0; i < n; ++i)
13    res[i] = round(val_A[i].real());
14  int carry = 0;
15  for(int i = 0; i < n; i++){ res[i] += carry;
16    carry = res[i] / 10; res[i] %= 10;
17  } return res;
18 }
```

FORMULAZAS

► Lógica - Conjuntos - Bitwise

⊕ es el xor (o diferencia simétrica de conjuntos).

- 1. $a|b = a \oplus b + a\&b$
- 2. $a \oplus (a\&b) = (a|b) \oplus b$
- 3. $b \oplus (a\&b) = (a|b) \oplus a$
- 4. $(a\&b) \oplus (a|b) = a \oplus b$
- 5. $a + b = a|b + a\&b = a \oplus b + 2(a\&b)$
- 6. $a - b = (a \oplus (a\&b)) - ((a|b) \oplus a) = ((a|b) \oplus b) - ((a|b) \oplus a)$
- 7. $a - b = (a \oplus (a\&b)) - (b \oplus (a\&b)) = ((a|b) \oplus b) - (b \oplus (a\&b))$

Si p, q son predicados, entonces:

- 1. $(p \vee q) \iff \neg(\neg p \wedge \neg q)$.
- 2. $(p \implies q) \iff (\neg p \vee q)$.
- 3. $(p \oplus q) \iff \neg(p \iff q)$.

► Combinatoria

Inclusión-Exclusión. Sean $A_1, \dots, A_n$ conjuntos, entonces

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{\emptyset \neq J \subseteq [n]} (-1)^{|J|-1} \left| \bigcap_{j \in J} A_j \right|$$

Cantidad de elementos en exactamente r conjuntos. Sean $A_1, \dots, A_n$ conjuntos y $0 \leq r \leq n$, la cantidad de elementos en exactamente r conjuntos A_i es

$$\sum_{m=r}^n (-1)^{m-r} \binom{m}{r} \sum_{\substack{J \subseteq [n] \\ |J|=m}} \left| \bigcap_{j \in J} A_j \right|$$

Teorema del binomio generalizado. Para $x \in \mathbb{R}$ y $a, b \in \mathbb{C}$ se cumple

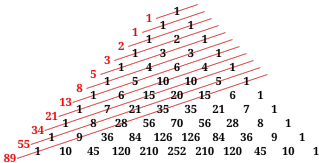
$$(a + b)^x = \sum_{k=0}^{\infty} \binom{x}{k} a^x b^{x-k},$$

$$\binom{x}{k} = \frac{1}{k!} \prod_{j=0}^{k-1} (x - j) = \frac{x(x-1)(x-2) \cdots (x-k+1)}{k!}.$$

ES FIBONACCI (*En un icpc, el de la Cangurera lo usó*). En el *Triángulo de Pascal*, los elementos en una “diagonal” satisfacen

$$F_{n+1} = \sum_{k=0}^{\lfloor n/2 \rfloor} \binom{n-k}{k},$$

donde F_{n+1} es el $(n+1)$ -ésimo número de Fibonacci.



Palo de Hockey. Para n y r ($n \geq r$), se cumple

$$\sum_{k=r}^n \binom{k}{r} = \binom{n+1}{r+1}.$$

Derangements. ¿Cuántos trastornos mentales distintos de tamaño n hay? (Según Google Translator). Considera los conjuntos $\{A_k\}_{k=1}^{k=n}$ donde A_k es la cantidad de permutaciones con el punto fijo $p_k = k$. La cantidad de permutaciones con al menos un punto fijo es

$$\begin{aligned} P &:= \left| \bigcup_{i=1}^n A_i \right| = \sum_{\emptyset \neq J \subseteq [n]} (-1)^{|J|-1} \left| \bigcap_{j \in J} A_j \right| \\ &= \sum_{\emptyset \neq J \subseteq [n]} (-1)^{|J|-1} (n - |J|)! = \sum_{i=1}^n (-1)^{i-1} \binom{n}{i} (n-i)!, \end{aligned}$$

entonces la cantidad de desarreglos es $n! - P$. Si d_n es la cantidad de desarreglos de tamaño n se cumple $d_n = (n-1)(d_{n-1} + d_{n-2})$ con $d_1 = 0$ y $d_0 = d_2 = 1$. También se cumple $d_n = n!d_{n-1} + (-1)^n$. Sea $0 \leq k \leq n$ y $d_{n,k}$ la cantidad de desarreglos de tamaño n con exactamente k puntos fijos, entonces $d_{n,k} = \binom{n}{k} d_{n-k}$.

► Geometría

Teorema de los círculos de Descartes. Si cuatro círculos son mutuamente tangentes de curvatura $k_i = 1/r_i$ el teorema dice:

$$(k_1 + k_2 + k_3 + k_4)^2 = 2 \left(k_1^2 + k_2^2 + k_3^2 + k_4^2 \right)$$

$$k_4 = k_1 + k_2 + k_3 \pm 2\sqrt{k_1 k_2 + k_2 k_3 + k_3 k_1}$$

Formula de Heron. Dado un trinangulo de lados a, b, c , escribimos su semiperimetro $s = \frac{a+b+c}{2}$, inradio r , y cuirunradio R .

$$\begin{aligned} Area &= \sqrt{s(s-a)(s-b)(s-c)} \\ 2R &= \frac{abc}{2\sqrt{s(s-a)(s-b)(s-c)}} \\ r &= \sqrt{\frac{(s-a)(s-b)(s-c)}{s}} \end{aligned}$$

Fórmula de Brahmagupta. Para el Area se vale algo similar en cuadrilateros ciclicos

$$Area = \sqrt{(s-a)(s-b)(s-c)(s-d)}$$

Para no cíclicos, θ = promedio de dos ángulos opuestos, p y q diagonales

$$\begin{aligned} Area &= \sqrt{(s-a)(s-b)(s-c)(s-d) - abcd \cos^2 \theta} \\ &= \sqrt{K}, \end{aligned}$$

donde $K = (s-a)(s-b)(s-c)(s-d) - \frac{1}{4}(ac+bd+pq)(ac+bd-pq)$. **Multidimensional** (*Como Chang*). Para más dimensiones, si A es la matriz con filas estos vectores

$$Volumen_n = \frac{1}{n!} \sqrt{\det(AA^T)}$$

MÉTODOS NUMÉRICOS

► Inversa de Matriz con Gauus-Jordan

Complejidad: Tiempo $O(n^3)$ - Memoria $O(n^2)$

```
1 #define eps 0.0001
2 vector<vector<double>> gauss(vector<vector<double>> m){
3   int tam = sz(m);
4   vector<vector<double>> id(tam, vector<double>(tam));
5   for(int i=0; i<tam; i++) id[i][i] = 1;
6   for(int i=0; i<tam; i++){
7     // buscar un no 0 para poner en (i,i)
8     if(abs(m[i][i]) < eps){
9       int j=i+1;
10      while(abs(m[j][i]) > eps && j<tam){
11        swap(m[j], m[i]);
12        swap(id[j], id[i]); j++;
13      }
14      if(j == tam){/*...*/} //no invertible
15    }
16    // poner un uno en (i,i) // modificar la fila i
17    double tmp = m[i][i];
18    for(int k=0; k<tam; k++){
19      id[i][k] /= tmp; m[i][k] /= tmp;
20      // poner columna i en todo 0, excepto casilla (i,i)
21      for(int j=0; j<tam; j++){
22        if(j==i) continue;
23        //a la fila j le restamos la fila i
24        tmp = m[j][i];
25        for(int k=0; k<tam; k++){
26          m[j][k] = m[j][k]-tmp*m[i][k];
27          id[j][k] = id[j][k]-tmp*id[i][k];
28        }
29      }
30    } return id;
31  }
32 vector<double> matporvec(vector<vector<double>> &m,
33   vector<double> &b){
34   vector<double> ans(sz(b));
35   for(int i=0; i<sz(b); i++){
36     for(int k=0; k<sz(b); k++){
37       ans[i] += m[i][k]*b[k];
38     } return ans;
39   }
```

► Sistemas de ecuaciones lineales

Dado un sistema de n ecuaciones con m incógnitas. Devuelve el numero de soluciones del sistema 0, 1 o ∞ y si existe al menos una solución la guarda en el vector ans. Complejidad: Tiempo $O((n+m)nm)$ - Memoria $O(nm)$.

```
1 const double EPS = 1e-9;
2 const int INF = 2; // it doesn't actually have to be infinity
  ↳ or a big number
3 int gauss(vector<vector<double>> a, vector<double> &ans){
4     int n = (int) sz(a);
5     int m = (int) sz(a[0]) - 1;
6     vi where(m, -1);
7     for(int col=0, row=0; col<m && row<n; ++col){
8         int sel = row;
9         for (int i=row; i<n; ++i)
10            if(abs(a[i][col])>abs(a[sel][col]))sel=i;
11        if(abs(a[sel][col]) < EPS) continue;
12        for(int i=col; i<m; ++i)
13            swap(a[sel][i], a[row][i]);
14        where[col] = row;
15        for(int i=0; i<n; ++i)
16            if(i != row){
17                double c = a[i][col] / a[row][col];
18                for(int j=col; j<m; ++j)
19                    a[i][j] -= a[row][j] * c;
20            } ++row;
21    } ans.assign(m, 0);
22    for(int i=0; i<m; ++i) if(where[i] != -1)
23        ans[i] = a[where[i]][m] / a[where[i]][i];
24    for(int i=0; i<n; ++i){
25        double sum = 0;
26        for(int j=0; j<m; ++j)
27            sum += ans[j] * a[i][j];
28        if(abs(sum - a[i][m]) > EPS) return 0;
29    }
30    for(int i=0; i<m; ++i) if(where[i] == -1)
31        return INF;
32    return 1;
33 }
```

► Sistemas de ecuaciones modulo 2

Dado un sistema de n ecuaciones con m incógnitas modulo 2. Devuelve el numero de soluciones del sistema 0, 1 o ∞ y si existe al menos una solución la guarda en el vector ans. Complejidad: Tiempo $O((n+m)nm)$ Memoria $O(nm)$.

```
1 int gauss (vector<bitset<MAXN>> A, int n, int m, bitset<MAXN>
  ↳ &ans){
2     // where[i] guarda el indice de la ecaucion donde se puede
  ↳ despejar variable i, -1 si es independiente
3     vi where(m, -1);
4     // Matriz "diagonal"
5     for(int col=0, row=0; col<m && row<n; ++col){
6         // busca pivot
7         for(int i = row; i<n; ++i) if(A[i][col]){
8             swap (A[i], A[row]);
9             break;
10        } if(!A[row][col]) continue;
11        where[col] = row;
12        // poner 0 todos lados
13        for(int i=0; i<n; ++i)
14            if(i != row && A[i][col]) A[i] ^= A[row];
15        ++row;
16    }
17    // sacar respuesta // variables independientes = 1, t
18    for(int i = 0; i < m; ++i)
19        ans[i] = where[i] == -1 ? 1 : ((A[where[i]].count() ^
  ↳ 1) & 1);
20    // si hay alguna variable libre
21    for(int i = 0; i < m; ++i) if(where[i] == -1)
22        return INF;
23    return 1;
24 }
```

SPARSE TABLE

Complejidad: Tiempo de precalculo $O(n \log n)$ - Tiempo en responder $O(\log(r-l+1))$ - Tiempo en responder para operaciones idempotentes $O(1)$ - Memoria extra $O(n \log n)$. LOGN es $\lceil \log_2(\text{MAXN}) \rceil$. Indexado en 0.

```
1 struct sparse_table{
2     int n, NEUTRO;
3     vvi ST;
4     vi lg2;
5     int f(int a, int b){return a + b;}
6     sparse_table(int _n, int data[]){
7         n = _n;
8         NEUTRO = 0;
9         lg2.resize(n + 1);
10        lg2[1] = 0;
11        for(int i=2; i<n; ++i)lg2[i]=lg2[i/2]+1;
12        ST.resize(lg2[n] + 1, vi(n + 1, NEUTRO));
13        for(int i=0; i<n; ++i)ST[0][i] = data[i];
14        for(int k = 1; k <= lg2[n]; ++k){
15            int fin = (1 << k) - 1;
16            for(int i = 0; i + fin < n; ++i)
17                ST[k][i]=f(ST[k-1][i],
18                           ST[k-1][i+(1<<(k-1))]);
19        }
20    }
21    int query(int l, int r){
22        if(l > r) return NEUTRO;
23        int ans = NEUTRO;
24        for(int k = lg2[n]; 0 <= k; --k){
25            if(r - l + 1 < (1 << k)) continue;
26            ans = f(ans, ST[k][l]);
27            l += 1 << k;
28        }
29        return ans;
30    }
31    int queryIdem(int l, int r){
32        if(l > r) return NEUTRO;
33        int lg = lg2[r - l + 1];
34        return f(ST[lg][l], ST[lg][r-(1<<lg)+1]);
35    }
36 };
```

FENWICK TREE

T: $O(\log n)$ | M: $O(n)$ | Indexado en 1

```
1 #define lsb(S) ((S) & (-S))
2 struct fenwick_tree{
3     int n;
4     vi BIT;
5     fenwick_tree(int _n): n(_n){BIT.resize(n + 1);}
6     void add(int pos, int x){
7         while(pos <= n){
8             BIT[pos] += x;
9             pos += lsb(pos);
10        }
11    }
12    int sum(int pos){
13        int res = 0;
14        while(pos){
15            res += BIT[pos];
16            pos -= lsb(pos);
17        } return res;
18    }
19 };
```

SEGMENT TREE

```
1 struct node{
2     int val, lazy;
3     node():val(0), lazy(0){}
4     node(int x, int lz = 0):val(x), lazy(lz){}
5     const node operator+(const node &b)const{
6         return node(val + b.val);
7     }
8 };
```

► Actualizaciones puntuales

Complejidad: Tiempo de precalculo $O(n)$ - Tiempo en responder $O(\log n)$ - Tiempo de actualización $O(\log n)$ - Memoria extra $O(n)$. Indexado en 1.

```
1 struct segment_tree{
2     struct node{...};
3     vector<node> nodes;
4     segment_tree(int n, int data[]){
5         nodes.resize(4 * n + 1);
6         build(1, n, data);
7     }
8     void build(int left, int right, int data[], int pos = 1){
9         if(left == right){
10            nodes[pos] = node(data[left]);
11            return;
12        }
13        int mid = (left + right) / 2;
14        build(left, mid, data, pos * 2);
15        build(mid + 1, right, data, pos * 2 + 1);
16        nodes[pos] = nodes[pos*2]+nodes[pos*2+1];
17    }
18    void update(int x, int idx, int left, int right, int pos =
  ↳ 1){
19        if(idx < left || right < idx) return;
20        if(left == right){
21            nodes[pos].val += x;
22            return;
23        }
24        int mid = (left + right) / 2;
25        update(x, idx, left, mid, pos * 2);
26        update(x, idx, mid+1, right, pos*2+1);
27        nodes[pos] = nodes[pos*2]+nodes[pos*2+1];
28    }
29    node query(int l, int r, int left, int right, int pos = 1){
30        if(r < left || right < l) return node();
31        if(l <= left && right <= r)
32            return nodes[pos];
33        int mid = (left + right) / 2;
34        return query(l, r, left, mid, pos*2) + query(l, r,
  ↳ mid+1, right, pos*2+1);
35    }
36 };
```

► Actualizaciones sobre rangos

Complejidad: Tiempo de precalculo $O(n)$ - Tiempo en responder $O(\log n)$ - Tiempo de actualización $O(\log n)$ - Memoria extra $O(n)$.

```
1 struct segment_tree{
2     struct node{...};
3     vector<node> nodes;
4     segment_tree(int n, int data[]){...}
5     void build(...){...}
6     void combine_lz(int lz, int pos){nodes[pos].lazy += lz;}
7     void apply_lz(int pos, int tam){
8         nodes[pos].val += nodes[pos].lazy * tam;
9         nodes[pos].lazy = 0;
10    }
11    void push_lz(int pos, int left, int right){
12        int len = abs(right - left + 1);
13        if(1 < len){
14            combine_lz(nodes[pos].lazy, pos*2);
15            combine_lz(nodes[pos].lazy, pos*2+1);
16        } apply_lz(pos, len);
17    }
18    void update(int x, int l, int r, int left, int right, int
  ↳ pos = 1){
19        push_lz(pos, left, right);
20        if(r < left || right < l) return;
21        if(l <= left && right <= r){
22            combine_lazy(x, pos);
23            push_lazy(pos, left, right);
```

```

24     return;
25 }
26 int mid = (left + right) / 2;
27 update(x, l, r, left, mid, pos * 2);
28 update(x, l, r, mid+1, right, pos*2+1);
29 nodes[pos] = nodes[pos*2]+nodes[pos*2+1];
30 }
31 node query(int l, int r, int left, int right, int pos = 1){
32     push_lz(pos, left, right);
33     ...
34 }
35 };

```

► Sparse segment tree

```

1 struct node{
2     ll lazy, maxi, sum;
3     node *left = nullptr, *right = nullptr;
4     node(): lazy(0), maxi(0), sum(0){}
5     void extend(){
6         if(left) return;
7         left = new node;
8         right = new node;
9     }
10    void combine_lz(ll lz){
11        lazy += lz;
12    }
13    void apply_lz(ll len){
14        sum += len * lazy; lazy = 0;
15    void push_lz(ll L, ll R){
16        int len = R - L + 1;
17        ll mid = L + (R - L) / 2;
18        if(len){
19            extend();
20            left->combine_lz(lazy);
21            right->combine_lz(lazy);
22        }
23        apply_lz(len);
24    }
25    void update(ll x, ll l, ll r, ll L, ll R){
26        push_lz(L, R);
27        if(r < L || R < l) return;
28        if(l <= L && R <= r){
29            combine_lz(x);
30            push_lz(L, R);
31            return;
32        }
33        ll mid = L + (R - L) / 2;
34        extend();
35        left->update(x, l, r, L, mid);
36        right->update(x, l, r, mid + 1, R);
37        sum = left->sum + right->sum;
38    }
39    ll query(ll l, ll r, ll L, ll R){
40        push_lz(L, R);
41        if(r < L || R < l) return 0;
42        if(l <= L && R <= r) return sum;
43        extend();
44        ll mid = L + (R - L) / 2;
45        return left->query(l, r, L, mid) + right->query(l, r,
46            mid + 1, R);
47    };

```

SQRT DECOMPOSITION

► Algoritmo de MO

Complejidad: Tiempo en responder $O((n+q)\sqrt{n}F + q \log q)$, donde $O(F)$ es la complejidad de `add()` y `remove()`.

```

1 const int block_size = 300;
2 struct query {
3     int l, r;
4     int block, i;
5     bool operator<(const query &b) const {
6         if(block == b.block) return r < b.r;
7         return block < b.block;
8     }

```

```

9 };
10 void add(int idx){/// TO-DO}
11 void remove(int idx){/// TO-DO}
12 int get_answer(){return 0; /// TO-DO}
13 vi solve(vector<query> &queries) {
14     vi answers(sz(queries));
15     sort(queries.begin(), queries.end());
16     int cur_l = 0, cur_r = -1;
17     for(query q : queries){
18         while(cur_l > q.l) add(--cur_l);
19         while(cur_r < q.r) add(++cur_r);
20         while(cur_l < q.l) remove(cur_l++);
21         while(cur_r > q.r) remove(cur_r--);
22         answers[q.i] = get_answer();
23     } return answers;
24 }

```

DSU

Complejidad: Tiempo $O(\log(n))$ - Memoria $O(n)$, donde n es la cantidad total de elementos. La complejidad temporal es por cada función.

P[MAXN]: guarda el representante para cada nodo.

RA[MAXN]: guarda el rango (peso) del conjunto de cada representante para el *small to large*.

```

1 struct dsu{
2     struct action{
3         int x_p, y_p, rank_y;
4     };
5     vi RA, P;
6     vector<action> actions;
7     dsu(int n){
8         RA.resize(n, 1);
9         P.resize(n);
10        iota(P.begin(), P.end(), 0);
11    }
12    int root(int x){
13        return x == P[x] ? x : P[x] = root(P[x]);
14    }
15    bool join(int x, int y, bool recording){
16        x = root(x);
17        y = root(y);
18        if(x == y) return false;
19        if(RA[x] >= RA[y]) swap(x, y);
20        if(recording) actions.pb({x, y, RA[y]});
21        RA[y] += RA[x];
22        P[x] = y;
23        return true;
24    }
25    void rollback(int cnt){
26        while(cnt-- > 0 && actions.size()){
27            action act = actions.back();
28            actions.pop_back();
29            RA[act.y_p] = act.rank_y;
30            P[act.x_p] = act.x_p;
31        }
32    }
33 };

```

GRAFOS

```

1 struct edge{
2     int from, to;
3     ll w;
4     const bool operator<(const edge &b) const{
5         return w > b.w;
6     }
7 };
8 struct pos{
9     int from;
10    ll c;
11    const bool operator<(const pos &b) const{
12        return c > b.c;
13    }
14 };

```

► Caminos mínimos

Dijkstra. Complejidad: Tiempo $O(|E|\log|V|)$ - Memoria extra $O(|E|)$.

```

1 ll dijkstra(int a, int b, vector<edge> graph[]){
2     ll dist[MAXN];
3     bool vis[MAXN] = {};
4     fill(dist, dist + MAXN, LLONG_MAX);
5     priority_queue<pos> q;
6     q.push(pos{a, 0}); dist[a] = 0;
7     while(sz(q)){
8         pos cur = q.top(); q.pop();
9         if(vis[cur.from]) continue;
10        vis[cur.from] = true;
11        for(edge &e : graph[cur.from]){
12            if(dist[e.to] <= dist[e.from] + e.w) continue;
13            dist[e.to] = dist[e.from] + e.w;
14            q.push(pos{e.to, dist[e.to]});
15        }
16    } return dist[b];
17 }

```

Bellman-Ford. Complejidad: $O(|V||E|)$.

```

1 vi bellman_ford(int s, int n, vector<edge> &edges, bool cycles
2     ⇨ = false){
3     vi d(n, (cycles ? 0 : INT_MAX));
4     d[s] = 0;
5     vi P(n, -1); /// Predecesor
6     for(int i = 0; i < n - 1; ++i)
7         for(edge &e : edges){
8             if(d[e.from] == INT_MAX) continue;
9             if(d[e.to] > d[e.from] + e.w){
10                d[e.to] = d[e.from] + e.w;
11                P[e.to] = e.from;
12            }
13        }
14    int last_relax = -1;
15    for(edge &e : edges){
16        if(d[e.from] == INT_MAX) continue;
17        if(d[e.to] > d[e.from] + e.w){
18            d[e.to] = d[e.from] + e.w;
19            P[e.to] = e.from;
20            last_relax = e.to;
21        }
22    }
23    if(last_relax == -1) return d;
24    return {}; /// VACIO

```

Floyd-Warshall. Complejidad: $O(|V|^3)$.

```

1 vvi floyd_warshall(int n){
2     const int INF = INT_MAX; /// para evitar overflow después
3     vvi d(n, vi(n, INF));
4     /// aquí inicializa con la lista/matriz de adyacencia
5     /// luego calcula la dp
6     for(int k = 0; k < n; ++k){
7         for(int i = 0; i < n; ++i){
8             for(int j = 0; j < n; ++j){
9                 if(d[i][k] == INF || d[k][j] == INF)
10                    continue;
11                 if(d[i][j] > d[i][k] + d[k][j]) d[i][j] =
12                    ⇨ d[i][k] + d[k][j];
13            }
14        } return d;
15 }

```

Johnson's algorithm. Complejidad: $O(|V||E|\log|V|)$. Sea $p : V \rightarrow \mathbb{R}$ una función potencial del grafo. El algoritmo es como sigue:

1. Hacemos una transformación en el grafo cambiando los pesos w a $w'(u, v) = w(u, v) + p(u) - p(v)$.

2. Calculamos la distancia mínima $d' : V \times V \rightarrow \mathbb{R}$ desde cada nodo a todos los demás con Dijkstra.

3. Finalmente, la distancia mínima de u a v en el grafo original es $d(u, v) = d'(u, v) - p(u) + p(v)$.

La función potencial p puede ser cualquiera. Usando Bellman-Ford se puede calcular el potencial $p(u)$ como el camino más corto que termina (o empieza) en u .

► Árboles

Prim. Complejidad: Tiempo $O(|E| \log |V|)$. `eCost[MAXN]` es el arreglo de costos mínimos de cada nodo para incluirlo en el MST.

```
1 ll prim(vector<edge> graph[]) {
2     ll e_cost[MAXN], ans = 0;
3     bool vis[MAXN] = {};
4     fill(e_cost, e_cost + MAXN, LLONG_MAX);
5     priority_queue<edge> q;
6     q.push(edge{1, 1, 0});
7     while(sz(q)) {
8         int node = q.top().to;
9         ll w = q.top().w; q.pop();
10        if(vis[node]) continue; vis[node] = true;
11        for(edge &e : graph[node]) {
12            if(vis[e.to] || e_cost[e.to] <= e.w) continue;
13            e_cost[e.to] = e.w;
14            q.push(e);
15        } ans += w;
16    } return ans;
17 }
```

Kruskal. Complejidad: Tiempo $O(|E| \log |E|)$.

```
1 ll kruskal(vector<edge> &edges, int n) {
2     sort(all(edges));
3     dsu mset(n);
4     ll res = 0;
5     for(edge &e : edges) {
6         if(mset.root(e.from) == mset.root(e.to)) continue;
7         mset.join(e.from, e.to);
8         res += e.w;
9     }
10    return res;
11 }
```

Boruvka. Complejidad: Tiempo $O(|E| \log |V|)$. $|V| = n$. `dsu.join()` devuelve true si la unión se llevó a cabo o false en otro caso.

```
1 ll boruvka(vector<edge> &edges, int n) {
2     dsu mset(n);
3     int min_edge[n];
4     ll res = 0;
5     while(mset.cnt_comp > 1) {
6         fill(min_edge, min_edge + n, -1);
7         for(int i = 0; i < sz(edges); ++i) {
8             int u = mset.root(edges[i].from);
9             int v = mset.root(edges[i].to);
10            if(u == v) continue;
11            if(min_edge[u] == -1 || edges[i].w <
12               ↪ edges[min_edge[u]].w) min_edge[u] = i;
13            if(min_edge[v] == -1 || edges[i].w <
14               ↪ edges[min_edge[v]].w) min_edge[v] = i;
15        }
16        for(int i = 0; i < n; ++i) {
17            int idx_e = min_edge[i];
18            if(idx_e == -1) continue;
19            res += mset.join(edges[idx_e].from,
20                           ↪ edges[idx_e].to) * edges[idx_e].w;
21        }
22    } return res;
23 }
```

MST dirigido. Complejidad: $O(|E| \log |V|)$. AGREGAR PEQUEÑA DESCRIPCIÓN.

```
1 /// MEJORAR ESTA COSA, SOLO LO COPIE Y
2   ↪ PEGUÉ por cuestiones de tiempo
3 const int N = 3e5 + 9;
4 const ll inf = 1e18;
5 template<typename T> struct PQ {
6     ll sum = 0;
7     priority_queue<T, vector<T>, greater<T>> Q;
8     void push(T x) { x.w -= sum; Q.push(x); }
9     T pop() { auto ans = Q.top(); Q.pop(); ans.w += sum;
10    ↪ return ans; }
11     int size() { return sz(Q); }
12     void add(ll x) { sum += x; }
13     void merge(PQ &x) {
14         if (size() < sz(x)) {
15             swap(sum, x.sum);
16             Q.swap(x.Q);
17         }
18         while(sz(x)) {
19             auto tmp = x.pop();
20             tmp.w -= sum;
21             Q.push(tmp);
22         }
23     };
24     struct edge {
25         int u, v; ll w;
26         bool operator > (const edge &rhs) const { return w >
27             ↪ rhs.w; }
28     };
29     struct DSU {
30         vi par;
31         DSU(int n) : par(n, -1) {}
32         int root(int i) { return par[i] < 0 ? i : par[i] =
33             ↪ root(par[i]); }
34         void set_par(int c, int p) { par[c] = p; }
35     };
36     // returns parents of each vertex
37     // each edge should be distinct
38     // it assumes that a solution exists (all vertices are
39     ↪ reachable from root)
40     // 0 indexed
41     // Takes ~300ms for n = 2e5
42     vi DMST(int n, int root, const vector<edge> &edges) {
43         vi u(2 * n - 1, -1), par(2 * n - 1, -1);
44         edge par_edge[2 * n - 1];
45         vi child[2 * n - 1];
46         PQ<edge> Q[2 * n - 1];
47         for(auto e : edges) Q[e.v].push(e);
48         for(int i = 0; i < n; i++) {
49             Q[(i+1) % n].push({i, (i+1) % n, inf});
50         }
51         int super = n;
52         DSU dsu(2 * n - 1);
53         int head = 0;
54         while(sz(Q[head])) {
55             auto x = Q[head].pop();
56             int nxt_root = dsu.root(x.u);
57             if (nxt_root == head) continue;
58             u[head] = nxt_root;
59             par_edge[head] = x;
60             if (u[nxt_root] == -1) head = nxt_root;
61             else {
62                 int j = nxt_root;
63                 do {
64                     Q[j].add(-par_edge[j].w);
65                     //Q[super].merge(move(Q[j]));
66                     Q[super].merge(Q[j]);
67                     assert(u[j] != -1);
68                     child[super].pb(j);
69                     j = dsu.root(u[j]);
70                 } while (j != nxt_root);
71                 for(auto u : child[super]) par[u] = super,
72                     ↪ dsu.set_par(u, super);
73                 head = super++;
74             }
75         }
76         vi res(2 * n - 1, -1);
77         queue<int> q; q.push(root);
78         while (sz(q)) {
```

```
73     int u = q.front();
74     q.pop();
75     while (par[u] != -1) {
76         for (auto v : child[par[u]]) {
77             if (v != u) {
78                 res[par_edge[v].v] = par_edge[v].u;
79                 q.push(par_edge[v].v);
80                 par[v] = -1;
81             }
82         }
83         u = par[u];
84     }
85 }
86 res[root] = root; res.resize(n);
87 return res;
88 }
89 int main() {
90     ios_base::sync_with_stdio(0); cin.tie(0);
91     int n, m, root; cin >> n >> m >> root;
92     vector<edge> edges(m);
93     for(auto &e : edges) cin >> e.u >> e.v >> e.w;
94     auto res = DMST(n, root, edges);
95     unordered_map<int, int> W[n];
96     for (auto u : edges) W[u.v][u.u] = u.w;
97     ll ans = 0;
98     for(int i = 0; i < n; i++) if(i != root)
99         ans += W[i][res[i]];
100    cout << ans << '\n';
101    for(auto x : res) cout << x << ' ';
102    cout << '\n';
103 }
```

Block-Cut Tree Complejidad: $O(n)$?

```
1 /// MEJORAR ESTA COSA, SOLO LO COPIE Y
2   ↪ PEGUÉ por cuestiones de tiempo
3 vvi comps; // nodes in each component
4 vvi bcc; // nodes to components that it belong
5 vi st; // internal stack
6 vi low, num;
7 int id;
8 int sz;
9 void dfs_biconnected(vvi &g, int u, int pre) {
10     low[u] = num[u] = ++id;
11     st.pb(u);
12     for(auto v : g[u]) {
13         if(!num[v]) {
14             dfs_biconnected(g, v, u);
15             low[u] = min(low[u], low[v]);
16             if(low[v] >= num[u]) {
17                 int x;
18                 comps.pb(vi());
19                 do {
20                     x = st.back();
21                     st.pop_back();
22                     bcc[x].pb(sz);
23                     comps.back().pb(x);
24                 } while(x ^ v);
25                 bcc[u].pb(sz);
26                 comps.back().pb(u);
27                 sz++;
28             } else if(v != pre) low[u] = min(low[u], num[v]);
29         }
30     }
31     vi utoubt; // its component or its AP index
32     vb uisart; // u is AP?
33     vvi bt; // block cut tree
34     void generateBlockCutTree(vvi &g) {
35         int n = sz(g);
36         sz = id = 0;
37         bcc.resize(0); low.resize(0); num.resize(0);
38         bcc.resize(n); low.resize(n); num.resize(n);
39         dfs_biconnected(g, 0, 0);
40         bt.resize(sz);
41         utoubt.resize(0); utoubt.resize(n);
42         uisart.resize(0); uisart.resize(n);
43         for(int u = 0; u < n; u++) {
44             if(sz(bcc[u]) > 1) { // Articulation
```

```

45     utoubt[u] = sz++;
46     uisart[u] = true;
47     bt.pb(vi());
48     for(auto v: bcc[u]){
49         bt[utoubt[u]].pb(v);
50         bt[v].pb(utoubt[u]);
51     }
52 } else //Not articulation point
53     utoubt[u] = bcc[u][0];
54 }
55 }

```

LCA. Complejidad: Tiempo de preproceso $O(|V| \log |V|)$. Tiempo de LCA y n -ésimo ancestro $O(\log |V|)$. 1-index.

```

1 void precalc(int node, int p = 0, int d = 1){
2     depth[node] = d;
3     P[0][node] = p;
4     for(int k = 1; k <= LOGN; ++k)
5         P[k][node] = P[k-1][P[k-1][node]];
6     for(int child : tree[node]) if(p != child)
7         precalc(child, node, d + 1);
8 }
9 int LCA(int a, int b){
10    if(depth[b] < depth[a]) swap(a, b);
11    int dif = depth[b] - depth[a];
12    for(int k = LOGN; 0 <= k; --k)
13        if(is_on(dif, k)) b = P[k][b];
14    if(a == b) return a;
15    for(int k = LOGN; 0 <= k; --k){
16        if(P[k][a] != P[k][b]){
17            a = P[k][a];
18            b = P[k][b];
19        }
20    }
21    return P[0][a];
22 }
23 int nth_ancestor(int u, int n){
24     for(int k = LOGN; 0 <= k; --k)
25         if(is_on(n, k)) u = P[k][u];
26     return u;
27 }

```

Sack. Complejidad: Tiempo $O(|V| \log |V|)$. 1-index.

```

1 void precalc(int node, int p = 0){
2     subtree_size[node] = 1;
3     depth[node] = depth[p] + 1;
4     for(int v : tree[node]){
5         if(v == p) continue;
6         precalc(v, node);
7         subtree_size[node] += subtree_size[v];
8     }
9 }
10 void add(int node, int x, int p = 0){
11     // add node here
12     // add subtree
13     for(int v : tree[node])
14         if(v != p && !big[v])
15             add(v, x, node);
16 }
17 void dfs(int node, bool keep, int p = 0){
18     int maxi = -1, big_child = -1;
19     for(int v : tree[node]) //Searchfor big_child
20         if(v != p && subtree_size[v] > maxi)
21             maxi = subtree_size[v], big_child = v;
22     for(int v : tree[node])
23         if(v != p && v != big_child)
24             dfs(v, false, node); // run a dfs on small childs
25                                     // and clear them
26     if(big_child != -1)
27         dfs(big_child, true, node), big[big_child] = 1; //
28                                     // big_child marked as big and not cleared
29     add(node, 1, p);
30     // answer queries here
31     if(big_child != -1) big[big_child] = 0;
32     if(!keep) add(node, -1, p);
33 }

```

► Máximo flujo

Algunos problemas de flujos

- *Maximum Weight Closure.* Sea N_1 una clausura de G y $N_2 = V \setminus N_1$, tenemos que $w(N_1) = \sum_{i \in N_1^+} w_i - \sum_{i \in N_1^-} |w_i|$ y $Cap.Corte = \sum_{i \in N_2^+} w_i + \sum_{i \in N_1^-} |w_i|$. Entonces

$$Cap.Corte + w(N_1) = \sum_{i \in N_1^+} w_i + \sum_{i \in N_2^+} w_i.$$

- *Mínima cobertura de vértices.* En grafos generales es NP-Completo. En grafos bipartitos el máximo emparejamiento es igual al numero de vertices en la mínima cobertura. Para el problema con pesos en los nodos, unimos s a todos los nodos en L con capacidad igual al peso de cada nodo, unimos los nodos de R a t de la misma manera y unimos los nodos de L a R con capacidad infinita. El máximo flujo es el peso mínimo de la mínima cobertura.
- *Máximo conjunto independiente.* Cualquier conjunto independiente es el complemento de alguna cobertura de vértices.
- *Mínimo cubrimiento de caminos independientes.* En grafos generales es NP-hard. En DAG's duplicamos los nodos en un lado IN y un lado OUT. Conectamos s al lado OUT y el lado IN a t . Las aristas del DAG las agregamos del lado OUT al lado IN. Sea M el máximo emparejamiento de la red anterior, entonces el mínimo cubrimiento es $|V| - M$.
- *Mínimo cubrimiento de caminos NO necesariamente independientes.* En grafos generales es NP-hard. En DAG's transformamos el DAG a su clausura transitiva y aplicamos el problema anterior.
- *Teorema de Menger.* Sea G un grafo finito (sin pesos) con vértices s, t distintos. El corte mínimo (por aristas) entre s y t es igual a la cantidad máxima de caminos independientes por aristas de s a t . Si s y t no son adyacentes, el corte mínimo por vértices entre s y t es igual a la cantidad máxima de caminos independientes por vértices de s a t .
- *Teorema de Mirsky.* En todo POSET, el tamaño de la cadena de mayor tamaño es igual al número de anticadenas necesarias para cubrir todos los elementos del conjunto.
- *Teorema de Dilworth.* En todo POSET, el tamaño de la anticadena de mayor tamaño es igual al número de cadenas necesarias para cubrir todos los elementos del conjunto.
- *Teorema de Hall.* Un grafo bipartito con subconjuntos L y R tiene un emparejamiento de tamaño $|L|$ si y sólo si para todo subconjunto W de L , se cumple que $|W| \leq |N_G(W)|$, donde $N_G(W)$ es el conjunto de vértices vecinos de alguno de los vértices en W .

Edmonds-Karp Complejidad: Ford-Fulkerson $O(|E| \cdot F)$, Edmonds-Karp $O(|V||E|^2)$.

```

1 struct edge {
2     int from, to;
3     ll w, c, f;
4     // weight, capacity, flow
5 };
6 class ford_fulkerson {
7 public:
8     ford_fulkerson (vector<vector<edge>> &graph) :
9         graph(graph){}
10    ll get_max_flow(int s, int t){
11        init();
12        ll f = 0;
13        while(find_and_update(s, t, f)){}
14        return f;
15    }
16    vi get_st_cut(const int &s){
17        bool vis[sz(graph)] = {};
18        vi S;
19        queue<int> q;
20        q.push(s);
21        S.pb(s);
22        vis[s] = true;
23        while(sz(q)){
24            int u = q.front(); q.pop();
25            for(int eI : edge_indexes[u]){
26                if(edges[eI].c > edges[eI].f &&
27                    !vis[edges[eI].to]){
28                    q.push(edges[eI].to);
29                    vis[edges[eI].to] = true;
30                    S.pb(edges[eI].to);
31                }
32            }
33        }
34        return S;
35    }
36    vector<vector<edge>> get_residual_graph(){
37        vector<vector<edge>> residual(sz(graph));
38        for(int i=0; i<sz(edges); i+=2){
39            const edge& e = edges[i];
40            if(e.c > 0){
41                residual[e.from].pb({e.from, e.to, e.w, e.c -
42                    e.f, e.f});
43                residual[e.to].pb({e.to, e.from, -e.w, e.f,
44                    -e.f});
45            }
46        }
47        return residual;
48    }
49 private:
50     vector<vector<edge>> graph;
51     vector<edge> edges;
52     vvi edge_indexes;
53     void init(){
54         edges.clear();
55         edge_indexes.clear();
56         edge_indexes.resize(sz(graph));
57         for(int u = 0; u < sz(graph); u++){
58             for(edge &e : graph[u]){
59                 edges.pb({u, e.to, e.w, e.c, 0});
60                 edges.pb({e.to, u, -e.w, 0, 0});
61                 edge_indexes[u].pb(sz(edges)-2);
62                 edge_indexes[e.to].pb(sz(edges)-1);
63             }
64         }
65     }
66     bool find_and_update(int s, int t, ll &flow){
67         queue<int> q;
68         // Desde donde llego y con que arista
69         vpii from(sz(graph), {-1, -1});
70         q.push(s);
71         from[s] = {s, -1};
72         bool found = 0;
73         while(sz(q) && !found){
74             int u = q.front(); q.pop();
75             for(int eI : edge_indexes[u]){
76                 if(edges[eI].c > edges[eI].f &&

```

```

72     from[edges[eI].to].fi== -1){
73         from[edges[eI].to] = {u, eI};
74         q.push(edges[eI].to);
75         if(edges[eI].to == t) found=1;
76     }
77 }
78 }
79 if(!found) return false;
80 ll u_flow = LLONG_MAX;
81 int cur = t;
82 while(cur != s) {
83     u_flow = min(u_flow, edges[from[cur].se].c -
84     ↪ edges[from[cur].se].f);
85     cur = from[cur].fi;
86 }
87 cur = t;
88 while(cur != s){
89     edges[from[cur].se].f += u_flow;
90     edges[from[cur].se].l.f -= u_flow;
91     cur = from[cur].fi;
92 }
93 flow += u_flow;
94 return 1;
95 };

```

Dinic. Complejidad: $O(|V|^2|E|)$. CAMBIAR A LA VARIANTE CON DFS *Multi-path augmentation*.

```

1  const int MAXV = 32767; // 2^15 - 1
2  template<class T = ll> struct dinic{
3      dinic(short V){this->V = V;}
4      const static bool SCALING = 1;
5      bool sorted = 0;
6      short s, t, V;
7      int lim = 1; // Para escalado
8      const T INF = numeric_limits<T>::max();
9      short level[MAXV]; // distancia desde s
10     short ptr[MAXV]; // arista que va explorando
11     struct edge{
12         short to, rev;
13         T cap, flow, mcap;
14         bool operator<(const edge &b) const {return mcap >
15         ↪ b.mcap;}
16 };
17 vector<edge> adj[MAXV];
18 vi adj_cur[MAXV]; // aristas del grafo de nivel
19 void add_edge(short u, short v, T cap, bool is_directed =
20 ↪ 1){
21     if(u == v) return;
22     T add = (is_directed ? 0 : cap);
23     adj[u].pb({v, sz(adj[v]), cap, 0, cap+add});
24     adj[v].pb({u, sz(adj[u]) - 1, add, 0, cap+add});
25 }
26 void mysort(){
27     if(sorted) return;
28     sorted = 1;
29     for(int i = 0; i < V; ++i){
30         sort(all(adj[i]));
31         for(int j=0; j<sz(adj[i]); ++j)
32             adj[adj[i][j].to][adj[i][j].rev].rev = j;
33     }
34     bool bfs(){
35         for(int i = 0; i < V; ++i){
36             adj_cur[i].clear();
37             adj_cur[i].reserve(sz(adj[i]));
38         }
39         queue<short> q;
40         q.push(s);
41         fill(level, level + V, -1);
42         level[s] = 0;
43         while(sz(q)){
44             short u = q.front(); q.pop();
45             if(u == t) return 1;
46             for(int i=0; i < sz(adj[u]); ++i){
47                 edge &e = adj[u][i];
48                 if(e.mcap < lim) break;

```

```

48         if(level[e.to] == -1 && e.cap - e.flow >= lim){
49             level[e.to] = level[u] + 1;
50             adj_cur[u].pb(i);
51             q.push(e.to);
52         } else if(level[e.to] == level[u] + 1 && e.cap -
53         ↪ e.flow >= lim){
54             adj_cur[u].pb(i);
55         }
56     }
57     return 0;
58 }
59 T dfs(short u, T flow, vector<short> &S, bool save = 0){
60     if(save) S.pb(u);
61     if(u == t) return flow;
62     for(; ptr[u] < sz(adj_cur[u]); ++ptr[u]){
63         edge &e = adj[u][adj_cur[u][ptr[u]]];
64         if(T pushed = dfs(e.to, min(flow, e.cap - e.flow),
65         ↪ S, save)){
66             e.flow += pushed;
67             adj[e.to][e.rev].flow -= pushed;
68             if(e.cap - e.flow < lim) ptr[u]++;
69             return pushed;
70         }
71     }
72     return 0;
73 }
74 ll get_max_flow(short source, short sink){
75     s = source; t = sink;
76     mysort();
77     vector<short> S;
78     ll flow = 0;
79     lim = SCALING ? (1<<30) : 1;
80     for(; 0 < lim; lim >= 1){
81         while(bfs()){
82             memset(ptr, 0, sizeof(ptr));
83             while(T pushed = dfs(s, INF, S))
84                 flow += pushed;
85         }
86         return flow;
87     }
88     vector<short> get_st_cut(){
89         vector<short> S;
90         memset(ptr, 0, sizeof(ptr));
91         dfs(s, INF, S, true);
92         return S;
93     }
94 }
95 };

```

MCMF. Complejidad: $O(\min\{|E|^2|V| \log |V|, F|E| \log |V|\})$.

```

1  const int MAXV = (1 << 15) - 1;
2  template<class T = ll> struct mcmf{
3      mcmf(short V){this->V = V;}
4      short s, t, V;
5      const T INF = numeric_limits<T>::max();
6      vector<T> p;
7      struct edge{
8          short to, rev;
9          T cap, flow, mcap, w;
10         bool operator<(const edge &b) const {return mcap >
11         ↪ b.mcap;}
12 };
13 vector<edge> adj[MAXV];
14 void add_edge(short u, short v, T cap, T w, bool
15 ↪ is_directed = 1){
16     if(u == v) return;
17     T add = (is_directed ? 0 : cap);
18     adj[u].pb({v, sz(adj[v]), cap, 0, cap + add, w});
19     adj[v].pb({u, sz(adj[u]) - 1, add, 0, cap + add, -w});
20 }
21 struct pos{
22     short from;
23     T c;
24     const bool operator<(const pos &b) const {return c > b.c;}
25 };
26 vector<T> bellman_ford(){
27     vector<T> d(V);
28     for(int i = 0; i < V - 1; ++i){
29         for(int u = 0; u < V; ++u){
30             for(edge &e : adj[u]){

```

```

29         if(d[e.to] > d[u] + e.w && e.cap > 0){
30             d[e.to] = d[u] + e.w;
31         }
32     }
33     return d;
34 }
35 pair<T, T> dijkstra(const T MAX_FLOW){
36     vector<T> d(V, INF);
37     vi P(V, -1), P_e(V, -1);
38     priority_queue<pos> q;
39     q.push({s, 0});
40     d[s] = 0;
41     while(sz(q)){
42         pos act = q.top(); q.pop();
43         if(act.c != d[act.from]) continue;
44         for(int j=0; j<sz(adj[act.from]); ++j){
45             edge &e = adj[act.from][j];
46             if(e.cap - e.flow <= 0) continue;
47             T w = e.w + p[act.from] - p[e.to];
48             if(d[e.to] <= d[act.from] + w)
49                 continue;
50             d[e.to] = d[act.from] + w;
51             q.push(pos{e.to, d[e.to]});
52             P[e.to] = act.from;
53             P_e[e.to] = j;
54         }
55     }
56     for(int i = 0; i < V; ++i) if(d[i] < INF)
57         d[i] += -p[s] + p[i];
58     for(int i = 0; i < V; ++i) if(d[i] < INF)
59         p[i] = d[i];
60     if(P[t] == -1) return {0, 0};
61     T flow = MAX_FLOW;
62     int cur_node = t;
63     while(cur_node != s){
64         flow = min(flow, adj[ P[cur_node] ][ P_e[cur_node]
65         ↪ ].cap - adj[ P[cur_node] ][ P_e[cur_node]
66         ↪ ].flow);
67         cur_node = P[cur_node];
68     }
69     T new_cost = 0;
70     cur_node = t;
71     while(cur_node != s){
72         adj[ P[cur_node] ][ P_e[cur_node] ].flow += flow;
73         new_cost += adj[ P[cur_node] ][ P_e[cur_node] ].w
74         ↪ * flow;
75         adj[cur_node][adj[ P[cur_node] ][ P_e[cur_node]
76         ↪ ].rev].flow -= flow;
77         cur_node = P[cur_node];
78     }
79     return {flow, new_cost};
80 }
81 pair<T, T> get_max_flow(short source, short sink, const T
82 ↪ MAX_FLOW = 1e8){
83     s = source;
84     t = sink;
85     p = bellman_ford();
86     T flow = 0, cost = 0;
87     while(flow < MAX_FLOW){
88         pair<T, T> pushed = dijkstra(MAX_FLOW - flow);
89         if(!pushed.fi) break;
90         flow += pushed.fi;
91         cost += pushed.se;
92     }
93     return {flow, cost};
94 }
95 };

```

► SCC

Kosajaru Complejidad: Tiempo $O(n)$.

```

1  void dfs(int u, vi graph[], bool vis[], vi &topo_ord){
2      if(vis[u]) return;
3      vis[u] = true;
4      for(int v : graph[u]) dfs(v, graph, vis, topo_ord);

```

```

5     topo_ord.pb(u);
6 }
7 void assign_scc(int u, vi inv_graph[], bool vis[], vi &scc,
    ↪ const int id){
8     if(vis[u]) return;
9     vis[u] = true;
10    scc[u] = id;
11    for(int v : inv_graph[u])
12        assign_scc(v, inv_graph, vis, scc, id);
13 }
14 pair<int, vi> kosajaru(int n, vi graph[], vi inv_graph[]){
15     bool vis[n] = {};
16     vi topo_ord;
17     for(int i=0; i<n; ++i) dfs(i, graph, vis, topo_ord);
18     reverse(all(topo_ord));
19     memset(vis, 0, sizeof(vis));
20     vi scc(n);
21     int id = 0;
22     for(int u : topo_ord) if(!vis[u])
23         assign_scc(u, inv_graph, vis, scc, id++);
24     return {id, scc};
25 }
26 pair<vvi, vvi> build_scc_graph(int n, vi graph[], int max_scc,
    ↪ const vi &scc){
27     vvi scc_graph(max_scc), inv_scc_graph(max_scc);
28     for(int u = 0; u < n; ++u)
29         for(int v : graph[u])
30             if(scc[u] != scc[v]){
31                 scc_graph[scc[u]].pb(scc[v]);
32                 for(int u = 0; u < n_scc; ++u){
33                     sort(all(scc_graph[u]));
34                     auto it = unique(all(scc_graph[u]));
35                     scc_graph[u].resize(it - scc_graph[u].begin());
36                     for(int v : scc_graph[u])
37                         inv_scc_graph[v].pb(u);
38                 } return {scc_graph, inv_scc_graph};
39 }

```

► 2-Sat

Complejidad: Tiempo en responder $O(n)$.

```

1 struct two_sat{
2     int n;
3     vvi graph, inv_graph;
4     vi scc, ans;
5     vector<bool> vis;
6     two_sat(){}
7     two_sat(int _n){
8         n = _n;
9         graph.resize(2 * n);
10        inv_graph.resize(2 * n);
11        scc.resize(2 * n);
12        vis.resize(2 * n);
13        ans.resize(n);
14    }
15    void add_edge(int u, int v){
16        graph[u].pb(v);
17        inv_graph[v].pb(u);
18    }
19    /// al menos una es verdadera
20    void add_or(int p, bool val_p, int q, bool val_q){
21        add_edge(p+(val_p? n:0), q+(val_q? 0:n));
22        add_edge(q+(val_q? n:0), p+(val_p? 0:n));
23    }
24    /// exactamente una es verdadera
25    void add_xor(int p, bool val_p, int q, bool val_q){
26        add_or(p, val_p, q, val_q);
27        add_or(p, !val_p, q, !val_q);
28    }
29    /// p y q tienen el mismo valor
30    void add_and(int p, bool val_p, int q, bool val_q){
31        add_xor(p, !val_p, q, val_q);
32    }
33    /// Kosajaru
34    void dfs(int node, vi &topo_ord){...}
35    void assign_scc(int node, const int id){...}
36    /// construye respuesta
37    bool build_ans(){

```

```

38        fill(all(vis), false);
39        vi topo_ord;
40        for(int i=0; i<2*n; ++i)dfs(i, topo_ord);
41        fill(all(vis), false);
42        reverse(all(topo_ord));
43        int id = 0;
44        for(int u : topo_ord) if(!vis[u])
45            assign_scc(u, id++);
46        for(int i = 0; i < n; ++i){
47            if(scc[i] == scc[i + n]) return 0;
48            ans[i] = (scc[i]<scc[i + n] ? 0:1);
49        } return 1;
50    }
51 };

```

TREAP

Complejidad: Tiempo $O(\log(n))$ - Memoria $O(n)$. Para treap implícito (arreglo dinamico) cambiar en insert/erase a split_by_pos().

```

1 struct treap{
2     typedef struct _node{
3         ll x;
4         int freq, cnt;
5         ll p;
6         _node *l, *r;
7         _node(ll _x): x(_x), p(((ll)(rand()) << 32)^rand()),
8             cnt(1), freq(1), l(nullptr), r(nullptr){}
9         ~_node(){delete l; delete r;}
10        void recalc(){
11            cnt = freq;
12            cnt += (l ? (l->cnt) : 0);
13            cnt += (r ? (r->cnt) : 0);
14        }
15    } * node;
16    node root;
17    node merge(node l, node r){
18        if(!l || !r) return l ? l : r;
19        if(l->p < r->p){
20            r->l = merge(l, r->l);
21            r->recalc();
22            return r;
23        } else {
24            l->r = merge(l->r, r);
25            l->recalc();
26            return l;
27        }
28    }
29    void split_by_value(node n, ll d, node &l, node &r){
30        l = r = nullptr;
31        if(!n) return;
32        if(n->x < d){
33            split_by_value(n->r, d, n->r, r);
34            l = n;
35        } else {
36            split_by_value(n->l, d, l, n->l);
37            r = n;
38        }
39        n->recalc();
40    }
41    void split_by_pos(node n, int pos, node &l, node &r, int
    ↪ l_nodes = 0){
42        l = r = NULL;
43        if(!n) return;
44        int cur_pos = (n->l ? (l_nodes + n->l->cnt) : l_nodes;
45        if(cur_pos < pos){
46            split_by_pos(n->r, pos, n->r, r, cur_pos + 1);
47            l = n;
48        } else {
49            split_by_pos(n->l, pos, l, n->l, l_nodes);
50            r = n;
51        }
52        n->recalc();
53    }
54    treap(): root(NULL){}
55    void insert_value(ll x){
56        node l, m, r;
57        split_by_value(root, x, l, m);
58        split_by_value(m, x + 1, m, r);

```

```

59        if(m){
60            m->freq++;
61            m->cnt++;
62        } else m = new _node(x);
63        root = merge(merge(l, m), r);
64    }
65    void erase_value(ll x){
66        node l, m, r;
67        split_by_value(root, x, l, m);
68        split_by_value(m, x + 1, m, r);
69        if(!m || m->freq == 1){
70            delete m;
71            m = nullptr;
72        } else {
73            m->freq--;
74            m->cnt--;
75        }
76        root = merge(merge(l, m), r);
77    }
78 };

```

STRINGS

► Hashing

T: $O(|s|)$ | **M:** $O(|s|)$ | Calcula el hash de los prefijos de un string. Admite concatenación y potencias.

hashing.cpp

```

1 ll bin_exp( ll a, ll exp, ll mod);
2 ll inv( ll b , ll mod);
3 ll div( ll a, ll b, ll mod);
4 struct mystring{
5     const ll A = 911382323;
6     const ll B = 972663749;
7     string s;
8     ll hash_val = -1, len = 0;
9     mystring(string &s): s(s){
10         len = sz(s);
11         hash_precalc();
12     }
13     mystring(ll hash_val, ll len):hash_val(hash_val), len(len){}
14     bool operator==(const mystring &o)const{
15         return (hash_val == o.hash_val) && (len == o.len);
16     }
17     bool operator!=(const mystring &other)const{
18         return !(*this == other);
19     }
20     vi h, powA;
21     ll hash_precalc(){
22         if(hash_val != -1) return hash_val;
23         if(len == 0) return hash_val = 0;
24         h.resize(len);
25         powA.resize(len + 1);
26         h[0] = s[0] % B;
27         powA[0] = 1;
28         for(int i = 1; i < sz(s); ++i){
29             h[i] = (h[i-1] * A + s[i]) % B;
30             powA[i] = A * powA[i-1] % B;
31         }
32         powA[len] = powA[len - 1] * A % B;
33         hash_val = h[sz(s) - 1];
34         return hash_val;
35     }
36     mystring hash_substring(ll i, ll j){
37         if(hash_val == -1) hash_precalc();
38         if(i < 0 || j >= len || i > j) return mystring(0, 0);
39         if(i == 0) return mystring(h[j], j + 1);
40         ll nhash = (h[j] - h[i-1] * powA[j - i + 1] % B + B) % B;
41         return mystring(nhash, j - i + 1);
42     }
43     mystring operator*(ll p)const{
44         if( p <= 0 || len == 0 ) return mystring(0, 0);
45         if( p == 1) return mystring(hash_val, len);
46         ll mult = bin_exp(A, len, B);
47         ll sum_geo;
48         if(mult == 1) sum_geo = p % B;
49         else{

```

```

50     ll num = (bin_exp(mult, p, B) - 1 + B) % B;
51     sum_geo = div(num, (mult - 1 + B) % B, B);
52 }
53 ll new_hash = hash_val * sum_geo % B;
54 ll new_len = len * p;
55 return mystring(new_hash, new_len);
56 }
57 mystring operator+(const mystring &o) const {
58     ll pA = bin_exp(A, o.len, B);
59     ll new_hash = (hash_val * pA % B + o.hash_val) % B;
60     ll new_len = len + o.len;
61     return mystring(new_hash, new_len);
62 }
63 };

```

► KMP

Complejidad: Tiempo $O(|s|)$ - Memoria extra $O(|s|)$.

```

1 vi prefix_function(const string &s){
2     int n = sz(s);
3     vi pi(n);
4     for(int i = 1; i < n; ++i){
5         int j = pi[i - 1];
6         while(j && s[i] != s[j]) j = pi[j - 1];
7         pi[i] = j + (s[i] == s[j]);
8     } return pi;
9 }

```

Autómata de KMP. Complejidad: Tiempo $O(|s|k)$ - Memoria extra $O(|s|k)$, donde k es el tamaño del alfabeto.

```

1 void compute_automaton(const string &s, vvi& aut){
2     s += '#';
3     int n = sz(s);
4     vi pi = prefix_function(s);
5     aut.assign(n, vi(26));
6     for(int i = 0; i < n; ++i){
7         for(int c = 0; c < 26; ++c){
8             if(i && 'a' + c != s[i])
9                 aut[i][c] = aut[pi[i - 1]][c];
10            else aut[i][c] = i + ('a' + c == s[i]);
11        }
12    }
13 }

```

► Suffix Automata

Complejidad: Tiempo $O(|s|)$, memoria $O(|S|)$. Construye un autómata sobre los sufijos de s , donde cada nodo terminal es un sufijo de s . Esta construcción se hace online. Donde α es el tamaño del alfabeto y $offset$ el primer caracter. Si se hace uso de los nodos terminales llamar a `markTerminalNodes()` primero (este método no es online).

```

1 constexpr short alpha = 26;
2 constexpr char offset = 'a';
3 struct state{
4     //max(state) = len, min(state) = sa[link].len
5     int len, link;
6     bool is_terminal;
7     array<int, alpha> next;
8     state(){
9         len = 0; link = -1; is_terminal = false;
10        next.fill(0);
11    }
12 };
13 struct SuffixAutomaton{
14     int n;
15     vector<state> sa;
16     int sz = 1, last = 0;
17     SuffixAutomaton(int n) : n(n), sa(2*n + 1){}
18     void add(char ch){
19         short ch = ch - offset;
20         int curr = sz++;
21         sa[curr].len = sa[last].len + 1;
22         int p = last;

```

```

23         while(p != -1 && !sa[p].next[ch]){
24             sa[p].next[ch] = curr;
25             p = sa[p].link;
26         }
27         if(p == -1){
28             sa[curr].link = 0;
29         } else {
30             int q = sa[p].next[ch];
31             if(sa[p].len + 1 == sa[q].len){
32                 sa[curr].link = q;
33             } else {
34                 int clone = sz++;
35                 sa[clone].len = sa[p].len + 1;
36                 sa[clone].next = sa[q].next;
37                 sa[clone].link = sa[q].link;
38                 while(p != -1 && sa[p].next[ch] == q){
39                     sa[p].next[ch] = clone;
40                     p = sa[p].link;
41                 }
42                 sa[q].link = sa[curr].link = clone;
43             }
44             last = curr;
45             return;
46         }
47     }
48     void markTerminalNodes(){
49         int curr = last;
50         while(curr){
51             sa[curr].is_terminal = true;
52             curr = sa[curr].link;
53         }
54         return;
55     }
56 };

```

► Suffix array

Construcción. Complejidad: Tiempo $O(|s| \log(|s|))$ - Memoria $O(|s|)$. Calcula la permutación que corresponde a los sufijos ordenados lexicográficamente. $SA[i]$ es el índice en el cual empieza el i -ésimo sufijo ordenado.

```

1 int SA[MAXN], mrank[MAXN];
2 int tmpSA[MAXN], tmp_mrank[MAXN];
3 void counting_sort(int k, int n){
4     int freqs[MAXN] = {0};
5     for(int i = 0; i < n; ++i){
6         if(i + k < n) freqs[mrank[i + k]]++;
7         else freqs[0]++;
8     }
9     int m = max(100, n);
10    for(int i = 0, sfs = 0; i < m; ++i){
11        int f = freqs[i];
12        freqs[i] = sfs;
13        sfs += f;
14    }
15    for(int i = 0; i < n; ++i){
16        if(SA[i] + k < n) tmpSA[mrank[SA[i] + k]]++
17        else tmpSA[0]++;
18    }
19    for(int i = 0; i < n; ++i) SA[i] = tmpSA[i];
20 }
21 void buildSA(string &str){
22     int n = sz(str);
23     for(int i = 0; i < n; ++i){
24         mrank[i] = str[i] - '#';
25         SA[i] = i;
26     }
27     for(int k = 1; k < n; k <= 1){
28         counting_sort(k, n);
29         counting_sort(0, n);
30         int r = 0;
31         tmp_mrank[SA[0]] = 0;
32         for(int i = 1; i < n; ++i){
33             if(mrank[SA[i]] != mrank[SA[i - 1]] || mrank[SA[i] + k] != mrank[SA[i - 1] + k])
34                 tmp_mrank[SA[i]] = ++r;

```

```

35         else tmp_mrank[SA[i]] = r;
36     }
37     for(int i = 0; i < n; ++i) mrank[i] = tmp_mrank[i];
38 }
39 }
40 inline bool suff_compare1(int idx, const string &pattern) {
41     return (s.substr(idx).compare(0, sz(pattern), pattern) < 0);
42 }
43 inline bool suff_compare2(const string &pattern, int idx) {
44     return (s.substr(idx).compare(0, sz(pattern), pattern) > 0);
45 }
46 pair<int, int> match(const string &pattern) {
47     int *low = lower_bound(SA, SA + sz(s), pattern,
48         suff_compare1);
49     int *up = upper_bound(SA, SA + sz(s), pattern,
50         suff_compare2);
51     return make_pair((int)(low - SA), (int)(up - SA));

```

Prefijo común más largo. Complejidad: Tiempo $O(|s|)$ - Memoria $O(|s|)$. Calcula la longitud del prefijo común más largo entre dos sufijos consecutivos (lexicográficamente) de s . $lcp[i]$ guarda la respuesta para el i -ésimo sufijo y el $(i - 1)$ -ésimo sufijo.

```

1 int lcp[MAXN];
2 void build_lcp(string &str){
3     int n = sz(str);
4     int phi[n];
5     phi[SA[0]] = -1;
6     for(int i = 1; i < n; ++i) phi[SA[i]] = SA[i - 1];
7     int plcp[n];
8     int k = 0;
9     for(int i = 0; i < n; ++i){
10        if(phi[i] == -1){
11            plcp[i] = 0;
12            continue;
13        }
14        while(i + k < n && phi[i] + k < n && str[i + k] ==
15            str[phi[i] + k]) k++;
16        plcp[i] = k;
17        k = max(k - 1, 0);
18    }
19    for(int i = 0; i < n; ++i) lcp[i] = plcp[SA[i]];

```

► Aho-Corasick

Construcción en $O(mk)$, donde m es el tamaño total de los strings y k el tamaño del alfabeto.

```

1 struct aho_corasick{
2     const static int K = 26;
3     const char index = 'a';
4     struct vertex{
5         int next[K];
6         bool terminal = false;
7         int p = -1;
8         char p_edge;
9         int link = -1;
10        int terminal_link = -1;
11        int go[K];
12        vertex(int p = -1, char c = '$') : p(p), p_edge(c){
13            memset(next, -1, K*sizeof(int));
14            memset(go, -1, K*sizeof(int));
15        }
16    };
17    vector<vertex> aho;
18    aho_corasick(){ aho.resize(1); }
19    void add_string(const string &s){
20        int u = 0;
21        for(char c : s){
22            int e = c - index;
23            if(aho[u].next[e] == -1){
24                aho[u].next[e] = sz(aho);
25                aho.emplace_back(u, c);
26            }

```

```

27     u = aho[u].next[e];
28     } aho[u].terminal = 1;
29 }
30 int get_link(int u){
31     if(aho[u].link == -1){
32         if(!u || !aho[u].p) aho[u].link = 0;
33         else aho[u].link = go(get_link(aho[u].p),
34                               ↪ aho[u].p_edge);
35     } return aho[u].link;
36 }
37 int go(int u, char c){
38     int e = c - index;
39     if(aho[u].go[e] == -1){
40         if(aho[u].next[e] != -1)
41             aho[u].go[e] = aho[u].next[e];
42         else aho[u].go[e] = !u ? 0 : go(get_link(u), c);
43     } return aho[u].go[e];
44 }
45 int get_terminal_link(int u){
46     if(aho[u].terminal_link == -1){
47         if(!u || !aho[u].p)
48             aho[u].terminal_link = 0;
49         else aho[u].terminal_link =
50             ↪ get_terminal_link(get_link(u));
51     } return aho[u].terminal_link;

```

► Suffix tree

```

1  /// MEJORAR ESTA COSA, SOLO LO COPIE Y
2  ↪ PEGUÉ por cuestiones de tiempo
3  const int inf = 1e9;
4  const int maxn = 1e6;
5  int s[maxn];
6  map<int, int> to[maxn];
7  //Root is the vertex 0
8  //f_pos[i] is the initial index with the letter of the edge
9  ↪ that goes from the parent of i to i
10 //len[i] is the number of letters in the edge that enters in i
11 //slink[i] is the suffix link
12 int len[maxn], f_pos[maxn], slink[maxn];
13 int node, pos;
14 int sz = 1, n = 0;
15 int make_node(int _pos, int _len){
16     f_pos[sz] = _pos;
17     len[sz] = _len;
18     return sz++;
19 }
20 void go_edge(){
21     while(pos > len[to[node][s[n - pos]]){
22         node = to[node][s[n - pos]];
23         pos -= len[node];
24     }
25 }
26 void add_letter(int c){
27     s[n++] = c;
28     pos++;
29     int last = 0;
30     while(pos > 0){
31         go_edge();
32         int edge = s[n - pos];
33         int &v = to[node][edge];
34         int t = s[f_pos[v] + pos - 1];
35         if(v == 0){
36             v = make_node(n - pos, inf);
37             //v = make_node(n - pos, 1);
38             slink[last] = node;
39             last = 0;
40         } else if(t == c) {
41             slink[last] = node;
42             return;
43         } else {
44             int u = make_node(f_pos[v], pos - 1);
45             to[u][c] = make_node(n - 1, inf);
46             to[u][t] = v;
47             f_pos[v] += pos - 1;
48             len[v] -= pos - 1;
49             v = u;

```

```

48     slink[last] = u;
49     last = u;
50 }
51 if(node == 0) pos--;
52 else node = slink[node];
53 }
54 }
55 void correct(int s_size){
56     len[0] = 0;
57     for (int i = 1; i < sz; i++){
58         if (f_pos[i] + len[i] - 1 >= s_size){
59             len[i] = (s_size - f_pos[i]);
60         }
61     }
62 }
63 void print_suffix_tree(int from){
64     cout << "Edge entering in " << from << " has size " <<
65     ↪ len[from];
66     cout << " and starts in " << f_pos[from] << endl;
67     cout << "Node " << from << " goes to: ";
68     for (auto u : to[from]){
69         cout << u.se << " with " << (char)u.fi << " ";
70     }
71     cout << endl;
72     for (auto u : to[from]){
73         print_suffix_tree(u.se);
74     }
75 }
76 void build(string &s){
77     for (int i = 0; i < sz; i++){
78         to[i].clear();
79     }
80     sz = 1;
81     node = pos = n = 0;
82     len[0] = inf;
83     for(int i = 0; i < sz(s); i++){
84         add_letter(s[i]);
85     }
86     correct(sz(s));
87 }
88 void cutGeneralized(vi &finishPoints){
89     for (int i = 0; i < sz; i++){
90         int init = f_pos[i];
91         int end = f_pos[i] + len[i] - 1;
92         int idx = lower_bound(all(finishPoints), init) -
93             ↪ finishPoints.begin();
94         if ((idx != sz(finishPoints)) && (finishPoints[idx] <=
95             ↪ end)){//Must be cut
96             len[i] = (finishPoints[idx] - f_pos[i] + 1);
97             to[i].clear();
98         }
99     }
100 }
101 void build_generalized(vector<string> &ss){
102     for (int i = 0; i < sz; i++){
103         to[i].clear();
104     }
105     sz = 1;
106     node = pos = n = 0;
107     len[0] = inf;
108     int sep = 256;
109     vi finishPoints;
110     int next = 0;
111     for (int i = 0; i < sz(ss); i++){
112         for (int j = 0; j < sz(ss[i]); j++){
113             add_letter(ss[i][j]);
114         }
115         next += sz(ss[i]);
116         finishPoints.pb(next);
117         add_letter(sep++);
118     }
119     correct(next);
120     cutGeneralized(finishPoints);

```

GEOMETRÍA

► Convex hull

Complejidad: $O(n \log n)$.

```

1  /// MEJORAR ESTA COSA, SOLO LO COPIE Y
2  ↪ PEGUÉ por cuestiones de tiempo
3  struct pt {
4      double x, y;
5  };
6  int orientation(pt a, pt b, pt c) {
7      double v = a.x*(b.y-c.y)+b.x*(c.y-a.y) + c.x*(a.y-b.y);
8      if(v < 0) return -1; // clockwise
9      if(v > 0) return +1; // counter-clockwise
10     return 0;
11 }
12 bool cw(pt a, pt b, pt c, bool include_collinear){
13     int o = orientation(a, b, c);
14     return o < 0 || (include_collinear && !o);
15 }
16 bool collinear(pt a, pt b, pt c){
17     return orientation(a, b, c) == 0;
18 }
19 void convex_hull(vector<pt> &a, bool include_collinear = 0){
20     pt p0 = *min_element(all(a), [](pt a, pt b){
21         return make_pair(a.y, a.x) < make_pair(b.y, b.x);
22     });
23     sort(all(a), [&p0](const pt& a, const pt& b){
24         int o = orientation(p0, a, b);
25         if(o == 0)
26             return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y)*(p0.y-a.y)
27                 < (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)*(p0.y-b.y);
28         return o < 0;
29     });
30     if(include_collinear){
31         int i = sz(a)-1;
32         while(i >= 0 && collinear(p0, a[i], a.back())) i--;
33         reverse(a.begin()+i+1, a.end());
34     }
35     vector<pt> st;
36     for(int i = 0; i < sz(a); i++){
37         while(sz(st)>1 && !cw(st[sz(st)-2], st.back(), a[i],
38             ↪ include_collinear))
39             st.pop_back();
40         st.pb(a[i]);
41     }
42     a = st;

```

UTILIDADES

► Subset sum optimization.

Complejidad: $O(N\sqrt{N})$. Si queremos obtener calcular el conjunto de sumas posibles en subconjuntos de un arreglo A , en el cual se cumple que $\sum a_i = N$ y todo $a_i \geq 0$, podemos obtener exactamente el mismo conjunto con un arreglo comprimido de tamaño $\sqrt{N}$. El $3k - trick$ explota que las sumas generadas por el arreglo $[a, a, a]$ son exactamente las mismas que en $[2a, a]$. Podemos juntar elementos y repetirlo hasta que a solo haya dos copias de cada elemento distinto en el nuevo arreglo, acotando el número de elementos a $O(\sqrt{N})$. El proceso de compresión toma $O(N \log_2(N))$.

```

1  vi compress(vector<ll> &a){
2      sort(a.rbegin(), a.rend());
3      int n = sz(a);
4      a.pb(0);
5      vi weights;
6      int pi = 0;
7      for(int i = 1; i <= n; i++){
8          if(a[i] != a[i - 1]){
9              ll cnt = i - pi;
10             ll x = a[i - 1];
11             ll j = 1;
12             while(j < cnt){

```

```

13     weights.pb(x * j);
14     cnt -= j;
15     j *= 2;
16 }
17 weights.pb(x * cnt);
18 pi = i;
19 }
20 } return weights;
21 }

```

► Bitsets de tamaño (casi) dinámico

El código ejecutado dentro de func(sz) usará un bitset con tamaño de la potencia de 2 más cercano a sz.

```

1 template<int len = 1> void func(int sz){
2     if(sz > len){
3         func<std::min(len * 2, MAXLEN)>(sz);
4         return;
5     }
6     bitset<len> bs;
7     //usa aquí tu bitset dinámico
8 }

```

► Bitwise ops

```

1 #define is_on(S, j) (S & (1ll << (j)))
2 #define set_bit(S, j) (S |= (1ll << (j)))
3 #define clear_bit(S, j) (S &= ~(1ll << (j)))
4 #define toggle_bit(S, j) (S ^= (1ll << (j)))
5 #define lsb(S) ((S) & -(S))
6 #define clear_lsb(S) (S &= (S - 1))
7 #define set_all(S, n) (S = (1ll << (n)) - 1ll)
8 #define clear_trailing_ones(S) (S &= (S + 1))
9 #define set_last_bit_off(S) (S |= (S + 1))
10 #define is_power_of_two(S) (!(S) & ((S) - 1))
11 #define nearest_power_of_two(S) ((int)pow(2,
12     ↪ (int)((log((double)(S)) / log(2)) + 0.5)) )
13 #define is_divisible_by_power_of_two(n, k) !((n) & ((1ll <<
14     ↪ (k)) - 1))
15 #define modulo(S, N) ((S) & ((N) - 1)) // S%N, N=2^k

```

► Bitwise - builtin

```

1 // one plus the index of the least significant 1-bit of x, or
2 ↪ if x is zero, returns zero.
3 int __builtin_ffs(int x)
4 // number of leading 0-bits in x, starting at the most
5 ↪ significant bit position. If x is 0 is undefined
6 int __builtin_clz(unsigned int x)
7 // number of trailing 0-bits in x, starting at the least
8 ↪ significant bit position. If x is 0 undefined.
9 int __builtin_ctz(unsigned int x)
10 // he parity of x, i.e. the number of 1-bits in x modulo 2.
11 int __builtin_parity(unsigned int x)

```

► Iterar

Dada una máscara m, iterar sobre todos sus subconjuntos desde el más grande hasta el más pequeño (ojo el código no itera sobre $x = m$). La complejidad de iterar sobre todas las submáscaras de todos los números de 1 a 2^n es $O(3^n)$.

```

1 void submasks(int mask){
2     for( int x = mask; x; ){
3         --x &= mask;
4     }
5 }

```

► Gossers' Hack

Sirve para generar todos las máscaras de n bits, que tengan exactamente k bits a 1 (y que sean menores o iguales que 2^n).

Complejidad $O\left(\binom{n}{k}\right)$?

```

1 void GossersHack(int n, int k) {
2     int mask = (1 << k) - 1;

```

```

3     int limit = (1 << n);
4     while(mask < limit){
5         int c = mask & -mask;
6         int r = mask + c;
7         mask = (((r ^ mask) >> 2) / c) | r;
8     }
9 }

```

► Subset Sum con bitset

Dado una una arreglo de tamaño n ver si es posible encontrar es la suma objetivo S . Nota: w es el tamaño del bloque de bits procesados en paralelo (normalmente 32 o 64). Complejidad: Temporal $O\left(n \times \frac{S}{w}\right)$ - Memoria $O(S)$

```

1 bool subsetSumBitset( vi& arr, int S) {
2     bitset<10001> dp; // tam >= S+1
3     dp[0] = 1;
4     for(int x : arr) dp |= (dp << x);
5     return dp[S];
6 }

```

► SOS DP

Dado un conjunto de n elementos y una función $A(S)$ definida para cada subconjunto S , queremos calcular

$$F(S) = \sum_{T \subseteq S} A(T), \quad \forall S \subseteq \{0, 1, \dots, n-1\}.$$

Definimos $S(x, i) = \{y \subseteq x \mid y \oplus x < 2^i\}$, notar que contiene todos los subconjuntos de x cuyos bits a la izquierda del bit i coinciden con los de x .

Ejemplo:

$$S(11010110_2, 3) = \{11010000, 11010010, 11010100, 11010110\}$$

Podemos descomponer $S(x, i)$ recursivamente:

$$S(x, i) = \begin{cases} S(x, i-1) & \text{si el bit } i \text{ de } x \text{ es } 0 \\ S(x, i-1) \cup S(x \oplus 2^i, i-1) & \text{si el bit } i \text{ de } x \text{ es } 1 \end{cases}$$

Con esta partición, definimos la tabla de DP:

$$dp[x][i] = \sum_{y \in S(x, i)} f[y]$$

Y así tenemos

$$dp[x][i] = \begin{cases} dp[x][i-1] & \text{si el bit } i \text{ de } x \text{ es } 0 \\ dp[x][i-1] + dp[x \oplus 2^i][i-1] & \text{si el bit } i \text{ de } x \text{ es } 1 \end{cases}$$

T: $O(n \cdot 2^n)$ — **M:** $O(2^n)$.

```

1 int n;
2 vector<int> f(1<<n); // inicializar f[S]
3 vector<int> g = f;
4
5 for(int i = 0; i < n; i++) {
6     for(int mask = 0; mask < (1<<n); mask++) {
7         if(mask & (1<<i)) {
8             g[mask] += g[mask^(1<<i)];
9         }
10    }
11 }

```

MÁXIMO DE FUNCIONES

► Li-Chao Tree

Dado un conjunto A con M valores a evaluar, y N funciones (tales que cada una de ellas se intersecta con el resto a lo más una

vez), te devuelve $\max_{i \in [N]} (f_i(a))$ para cualquier $a \in A$. Complejidad:

Responder y agregar $O(\log M)$.

```

1 struct Function {
2     ll m;
3     ll b;
4     ll eval(ll x){
5         if (m == LLONG_MIN) return LLONG_MIN;
6         return m*x+b;
7     }
8     Function(){ m = LLONG_MIN;}
9     Function(ll m_, ll b_): m(m_), b(b_){ }
10 };
11 struct LiChaoTree{
12     vll values;
13     ll maxV;
14     Function *functions;
15     LiChaoTree(vll &values_){
16         values = values_;
17         sort(all(values));
18         functions = new Function[sz(values)*4];
19         maxV = sz(values);
20     }
21     //Range from l to r - 1
22     ll get(ll x){
23         return get(x, 1, 0, maxV);
24     }
25     ll get(ll x, int v, int l, int r){
26         int m = l + (r - l) / 2;
27         ll mv = values[m];
28         if(r - l == 1){
29             return functions[v].eval(x);
30         } else if(x < mv){
31             return max(functions[v].eval(x), get(x, 2 * v, l,
32                 ↪ m));
33         } else {
34             return max(functions[v].eval(x), get(x, 2 * v + 1,
35                 ↪ m, r));
36         }
37     }
38     void addFunction(Function f){
39         addFunction(f, 1, 0, maxV);
40     }
41     void addFunction(Function f, int v, int l, int r){
42         int m = l + (r - l) / 2;
43         ll mv = values[m];
44         ll lv = values[l];
45         bool lef = f.eval(lv) > functions[v].eval(lv);
46         bool mid = f.eval(mv) > functions[v].eval(mv);
47         if(mid){//Si el actual pierde en el medio
48             swap(functions[v], f);
49         } if(r - l == 1) return;
50         else if(lef != mid){//El cruce esta en izq
51             addFunction(f, 2 * v, l, m);
52             else addFunction(f, 2 * v + 1, m, r);
53     }
54     ~LiChaoTree(){ delete[] functions; }
55 };

```